00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D

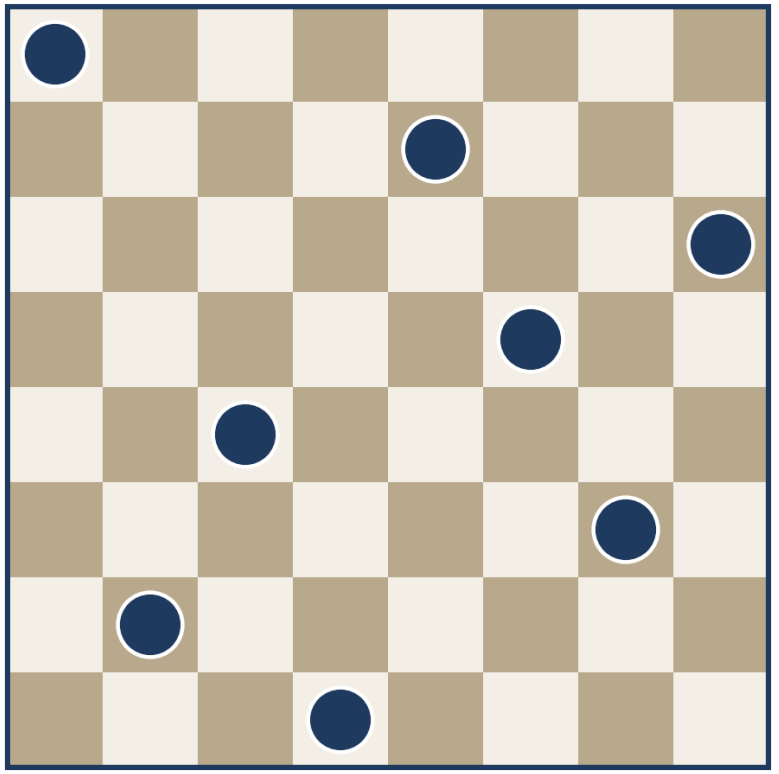
ALGORİTMA VE PROGRAMLAMA – II

HAFTA 03

Özyineleme ile Arama Uzayını Dolaşmak

Üretme Algoritmaları ve Geri İzleme

MODÜL I · ALGORİTMALARIN DİLİ



sütunlar = 0 4 7 5 2 6 1 3 (92 çözümün ilki)

Dr. Öğr. Üyesi Mehmet Sevri · RTEÜ Bilgisayar Mühendisliği · 2026–2027 Bahar

14 HAFTALIK YOLCULUK

Yol haritasında neredeyiz?

H1 Doğruluk ve Ölçüm	**H2** Asimptotik Analiz	**H3** **Özyineleme ve Geri İzleme**	**H4** Arama Algoritmaları	**H5** Temel Sıralama	**H6** Böl ve Fethet	**H7** Öbek ve Doğrusal Sıralama
0x00	0x01	0x02	0x03	0x04	0x05	0x06

H8 Dinamik Dizi ve Bağlı Liste	**H9** Yığın ve Kuyruk	**H10** Karma Tabloları	**H11** Graflar: BFS ve DFS	**H12** Açgözlü Algoritmalar	**H13** Dinamik Programlama	**H14** Sentez ve Final Projesi
0x07	0x08	0x09	0x0A	0x0B	0x0C	0x0D

Vize: H1–H7 · Ara proje: H7 (teslim: H8 başı) · Final projesi: H14

TAHMİN ET

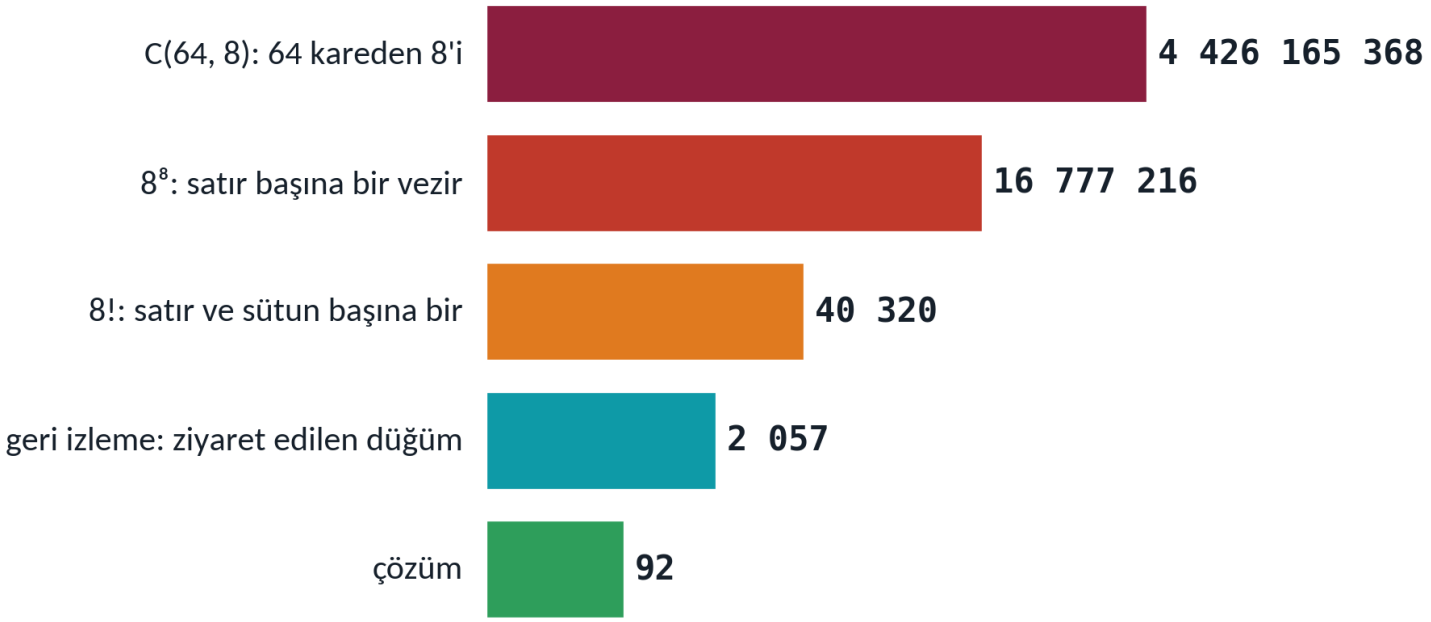
Geçen haftadan üç soru

- **S1 (H2).** $\Theta(n^2)$ bir algoritmada n iki katına çıkarsa süre kaç katına çıkar? $\Theta(2^n)$ bir algoritmada n yalnızca 1 artarsa?
- **S2 (AP-1 H9).** `toplam(n) = n + toplam(n – 1)`, `toplam(0) = 0`: `toplam(3)` çağrıldığında en derin anda yığında kaç çerçeve vardır?
- **S3 (AP-1 H4 ve H9).** 3 diskli Hanoi Kuleleri en az kaç hamle ister? C'de `1 << 3` kaçtır?

Yanıtlar: hepsi “ikiye katlanma”

- **S1:** $\Theta(n^2)$'de ≈ 4 kat. $\Theta(2^n)$'de n'nin 1 artması süreyi ≈ 2 katına çıkarır; n 10 artarsa ≈ 1024 kat.
- **S2:** 4 çerçeve: toplam(3), toplam(2), toplam(1), toplam(0). Örnek 1 ölçer: `toplam(5) = 15 çağrı = 6 en büyük derinlik = 6`
- **S3:** $2^3 - 1 = 7$ hamle (Örnek 2); `1 << 3` = 8.
- Bu hafta bu üç fikri birleştiriyoruz: **çağrı ağacı** (zaman), **yığın derinliği** (bellek), **2^n ve n!** büyüklüğünde arama uzayları.

Büyük Soru: 4,4 milyar yerleşim mi, 2 bin düğüm mü?



çubuk uzunluğu: log_{10} ölçek

- Satranç tahtasına birbirini tehdit etmeyen sekiz vezir kaç farklı biçimde yerleştirilebilir ve yaklaşık 4,4 milyar yerleşimi tek tek denemeden yanıtı nasıl buluruz?
- C(64, 8) = 4 426 165 368 yerleşim; satır başına bir vezirle 8^8 = 16 777 216; sütun kısıtıyla 8! = 40 320.
- Geri izleme 2 057 düğümde 92 çözümün hepsini bulur.

GERÇEK DÜNYADAN

1848 · 1850 · 1971

Sekiz vezir: bir satranç bulmacasından bir yöntemin adına

- Sekiz vezir bulmacası ilk kez 1848'de satranç problemcisi Max Bezzel tarafından soruldu; 92 çözüm 1850'de Franz Nauck tarafından yayımlandı. Niklaus Wirth, "adım adım iyileştirme" ile program geliştirmeyi anlattığı klasik makalesinde (Wirth, 1971) bu bulmacayı geri izleme algoritmasının örneği olarak kullandı.
- Wirth'ün makalesindeki iki sayı: program ilk çözümü bulmadan önce **876** kısmi yerleşim sınar; 92 çözümün hepsini kareleri **15 720** kez sınayarak bulur.
- Bu hafta kendi programlarımızın aynı iki sayıyı ölçtüğünü göreceğiz (Mühendis Gözüyle).

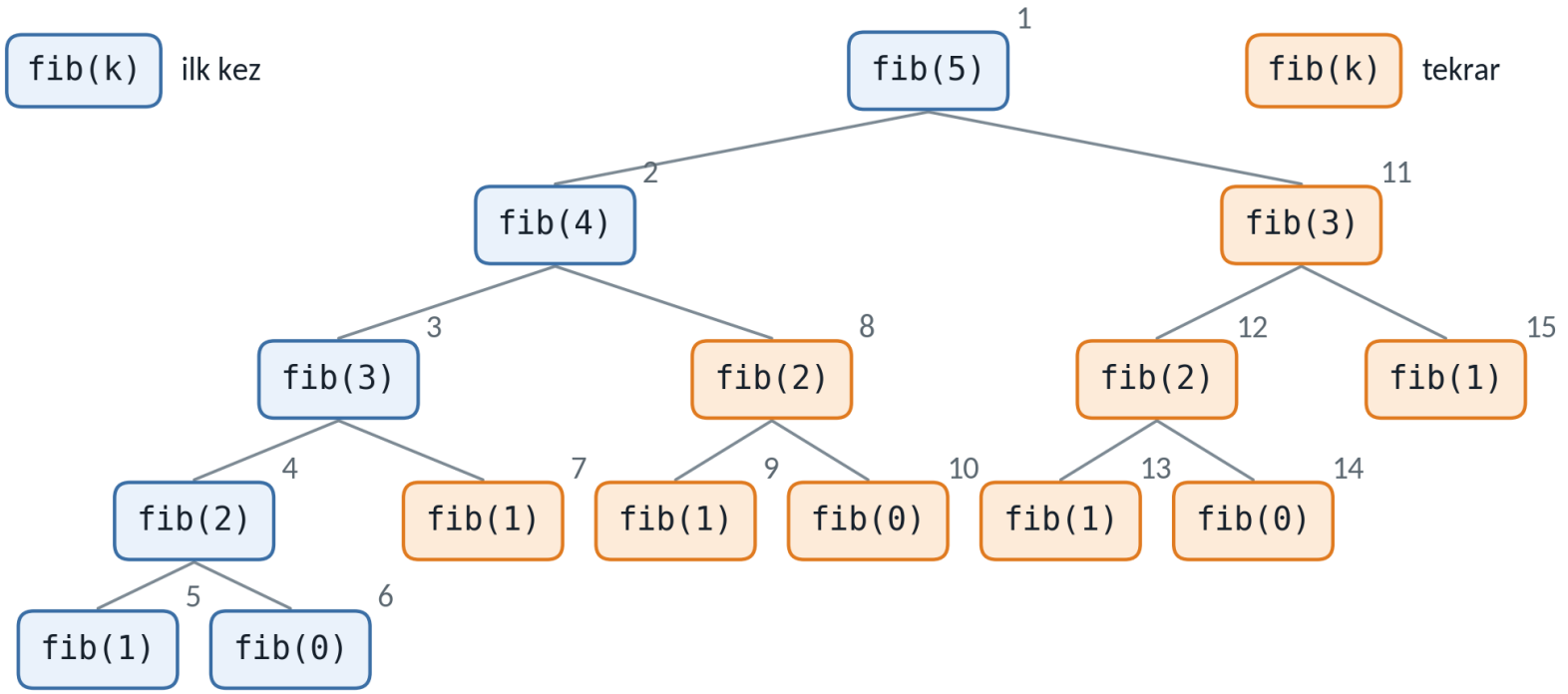
Wirth, N. (1971). Program development by stepwise refinement. Communications of the ACM, 14(4), 221–227.

ÖĞRENME ÇIKTILARI

Bu hafta sonunda neler yapabileceksiniz?

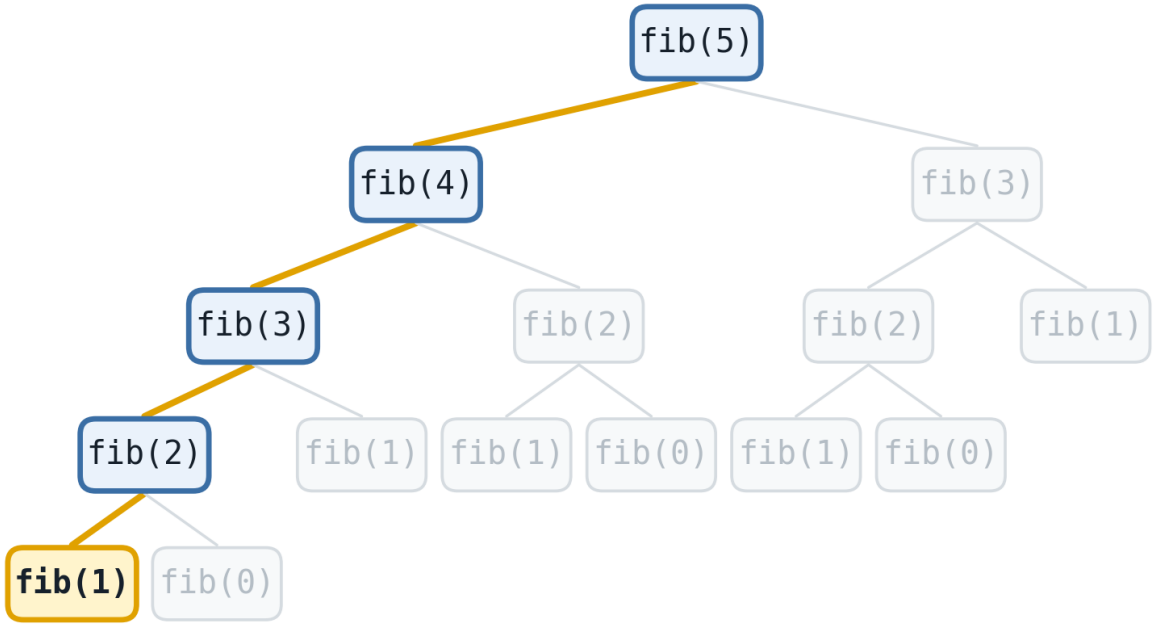
ÖÇ 3.1	Özyinelemeli bir algoritmayı özyineleme ağacıyla çizer; çağrı sayısını ve özyineleme derinliğini (bellek) belirler.
ÖÇ 3.2	Basit yineleme bağıntılarını — $T(n) = T(n-1) + c$, $T(n) = 2T(n-1) + c$, $T(n) = n \cdot T(n-1)$ — yerine koyma yöntemiyle çözer.
ÖÇ 3.3	Alt küme, permütasyon ve kombinasyonları özyinelemeyle ve bit maskesiyle üretir; üretilen nesne sayısını önceden hesaplar.
ÖÇ 3.4	Durum uzayı ağacı üzerinde geri izleme algoritmaları (N-vezir, alt küme toplamı, labirent) tasarlar; budama koşullarıyla arama uzayını daraltır.
ÖÇ 3.5	Kaba kuvvet çözümün uygulanabilirlik sınırını (2^n için $n \approx 25$, $n!$ için $n \approx 11$) hesaplar ve gerekçelendirir.

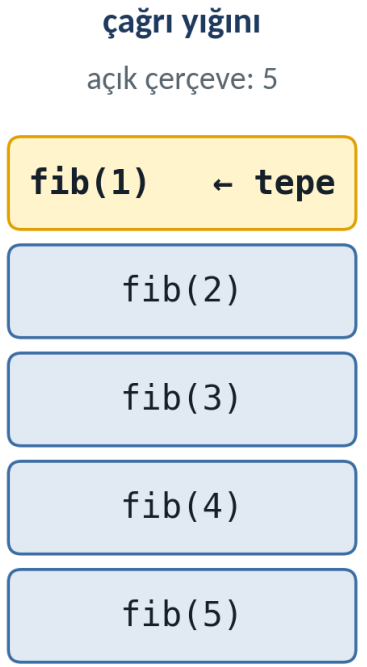
Özyineleme ağacı: fib(5) 15 çağrı, 6 farklı alt problem



- `fib(5) = 5 çağrı = 15 en büyük derinlik = 5`
- Turuncu düğüm: daha önce çözülmüş alt problemin **tekrarı** (9 çağrı).
- Düğüm sayısı → **zaman**; ağacın yüksekliği → **bellek** (yığın).
- C(n) = 2·F(n + 1) − 1: Örnek 1, n = 0 … 25 için denetler.

Aynı anın iki görünümü: kökten etkin düğüme giden yol = yığın





SİZCE SIRADA NE OLACAK?

fib(1) şimdi çalışıyor ve hemen dönecek. Ardından yığında hangi çerçeveler kalır ve hangi çağrı başlar?

Yığında aynı anda en çok **5** çerçeve: derinlik = n.

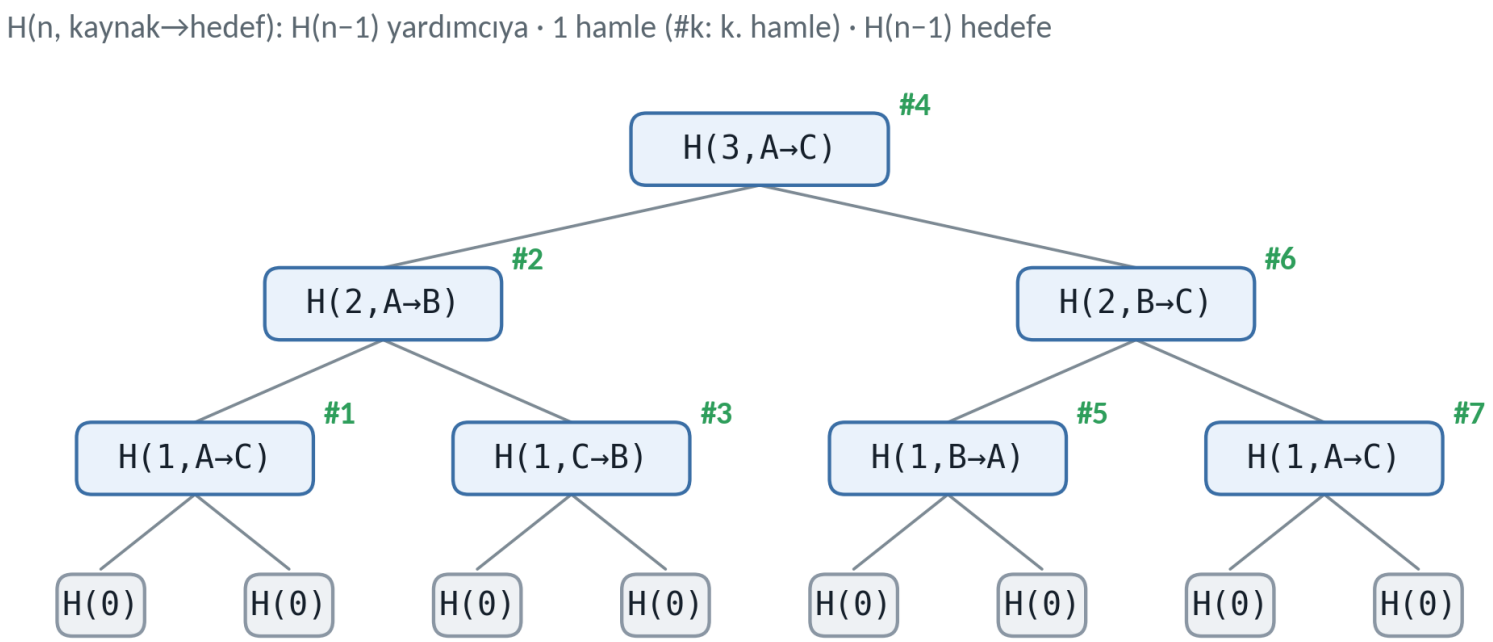
Yineleme bağıntıları: yerine koyma (açılım) yöntemi

$T(n) = T(n-1) + c$	$T(n) = 2T(n-1) + 1$	$T(n) = n \cdot T(n-1)$
faktöriyel (Kavram 3)	*Hanoi (Örnek 2)*	*permütasyon sayısı*
$T(n) = T(n-1) + c$ $= T(n-2) + 2c$ $= T(n-3) + 3c$ $= \dots$ $= T(n-k) + k \cdot c$ $k = n$: $T(0) + n \cdot c$	$T(n) = 2T(n-1) + 1$ $= 4T(n-2) + 3$ $= 8T(n-3) + 7$ $= \dots$ $= 2^k T(n-k) + 2^k - 1$ $k = n$: $2^n \cdot 0 + 2^n - 1$	$T(n) = n \cdot T(n-1)$ $= n(n-1) \cdot T(n-2)$ $= n(n-1)(n-2)T(n-3)$ $= \dots$ $k = n$: $n! \cdot T(0)$
$\Theta(n)$	$2^n - 1 = \Theta(2^n)$	$n!$ ($T(0) = 1$)

- **$T(n) = T(n-1) + c$** → $\Theta(n)$: faktöriyel, `toplam`.
- **$T(n) = 2T(n-1) + 1$** → $2^n - 1$: Hanoi, ikili diziler.
- **$T(n) = n \cdot T(n-1)$** → $n!$: permütasyon sayısı.
- Yöntem: birkaç kez açın, k adım için genel yazın, k'yı temel duruma ulaşacak seçin (Cormen vd., 2022).

ÖRNEK 2

Hanoi Kuleleri: T(n) = 2T(n − 1) + 1



- n − 1 diski yardımcıya, en büyüğü hedefe, n − 1 diski hedefe.
- n = 3: 7 hamle yapan iç düğüm + 8 boş H(0) çağrısı.
- k. hamle, ağacın **ortanca (in-order)** dolaşımındaki k. iç düğümde.
- n = 20: 1 048 575 hamle = $2^{20} - 1$ (sayaçla ölçüldü).

KOD

Hanoi: iki özyinelemeli çağrı, arada bir hamle

c/H03_02_hanoi.c · satır 16–28

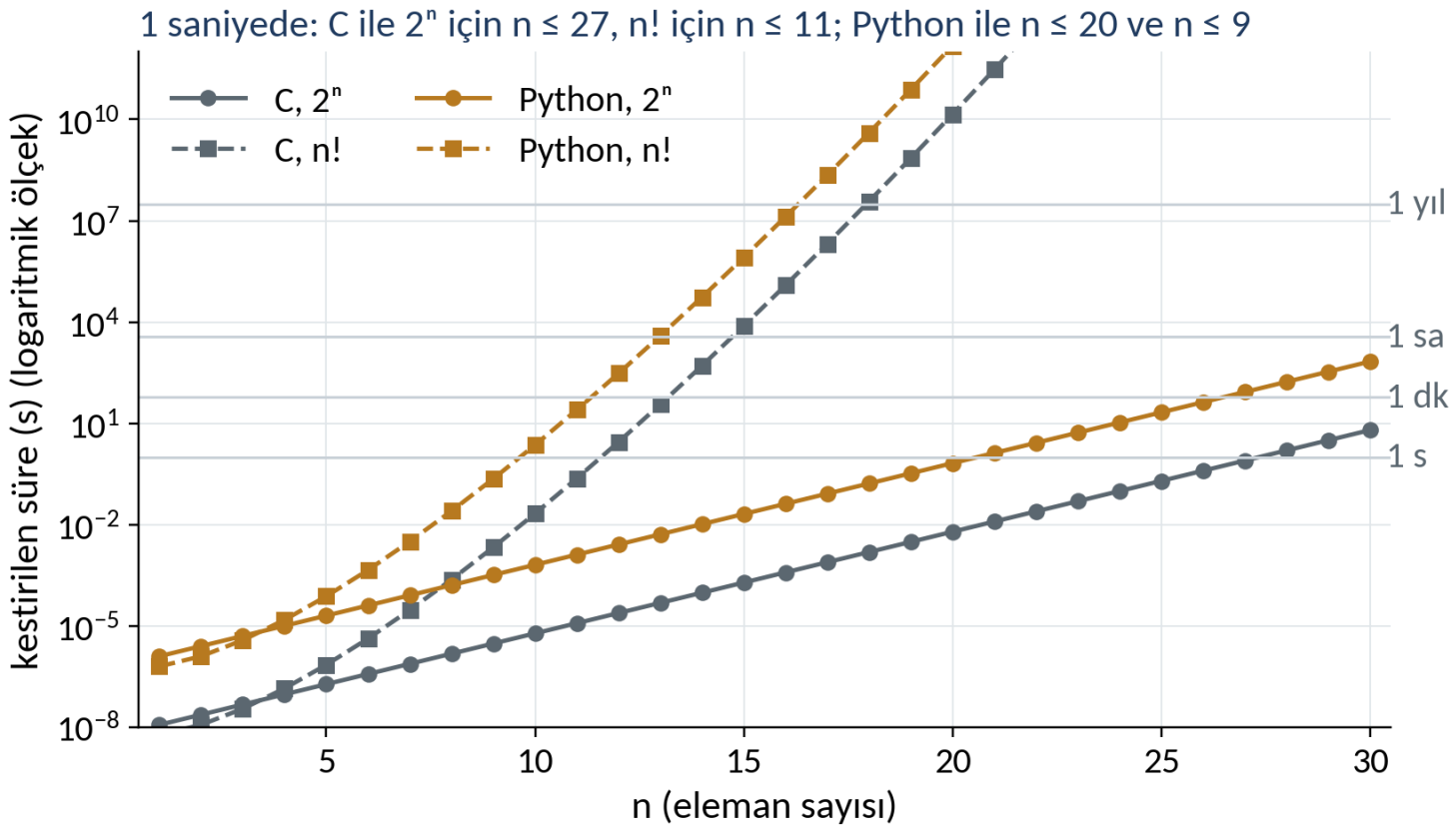
C

```c
static void hanoi(int n, char kaynak, char hedef, char yardimci)
{
    if (n == 0) {
        return;                                  /* temel durum: disk yok */
    }
    hanoi(n - 1, kaynak, yardimci, hedef);       /* üstteki n-1 disk */
    hamle_sayisi++;                              /* en büyük disk */
    if (listele) {
        printf("%2lld. hamle: disk %d  %c -> %c\n",
               hamle_sayisi, n, kaynak, hedef);
    }
    hanoi(n - 1, yardimci, hedef, kaynak);       /* n-1 disk yeniden */
}
```

ÇIKTI · İKİNCİ BÖLÜM (AÇILIM)

```
2) Yerine koyma: T(n) = 2T(n-1) + 1, T(0) = 0
T(5) = 2T(4) + 1 = 4T(3) + 3
     = 8T(2) + 7 = 16T(1) + 15
     = 32T(0) + 31 = 31
```

Uygulanabilirlik duvarı: 2^n ve $n!$



- 1 saniyede (bu makine, C -O2): 2^n için $n \leq 27$, $n!$ için $n \leq 11$.
- Python: $n \leq 20$ ve $n \leq 9$.
- 1 dakika sınırları yalnızca birkaç adım öteler (C: 33 ve 13).
- Çıktının kendisi üstelse (bütün alt kümeler) hiçbir budama işi polinoma indiremez: **$\Omega(n \cdot 2^n)$**.

Bir çağrı kaç bayt? Yığın nerede biter?

toplam(n) çerçevesi (gcc -O0)



32 bayt / çağrı

```
call · pushq %rbp
subq $16, %rsp
```

derinlik sınırları (bu makine, ulimit -s = 8192 KB)



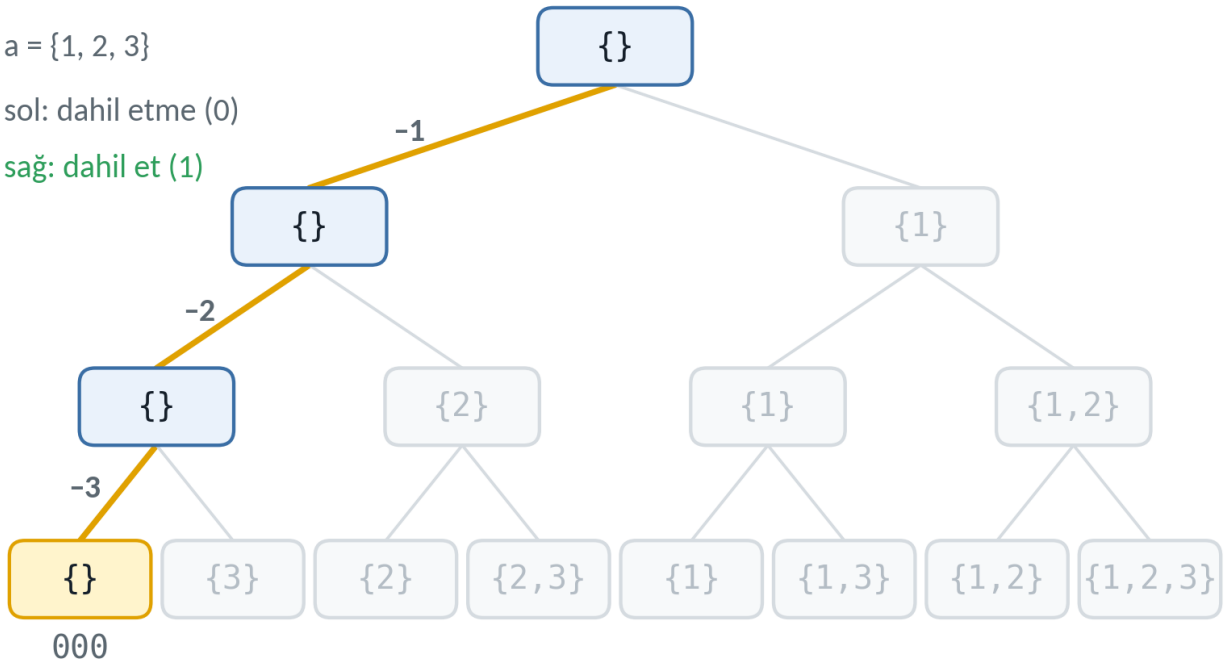
- gcc -O0: `toplam(n)` çerçevesi **32 bayt** (dönüş adresi + rbp + 16 B yerel).
- 8 MB yığın → ≈ 262 000 çerçeve; ilk çökme 261 213'te (Linux'ta tipik olarak SIGSEGV).
- -O2: GCC özyinelemeyi döngüye çevirir; n = 10^7 çalışır (standart güvence etmez).
- Python: sınır bir sayaçtır, `getrecursionlimit()` = 1000; 3.10'da sınırı yükseltmek C yığınını taşırdı.

Geri izleme şablonu: seç → dene → geri al



Üç şablon bu iskeletin özel durumu: dahil et / etme (2^n) · seç–dene–geri al ($n!$) · başlangıç indisi ($C(n, k)$).

Alt küme üretimi: her eleman için iki dal



1.

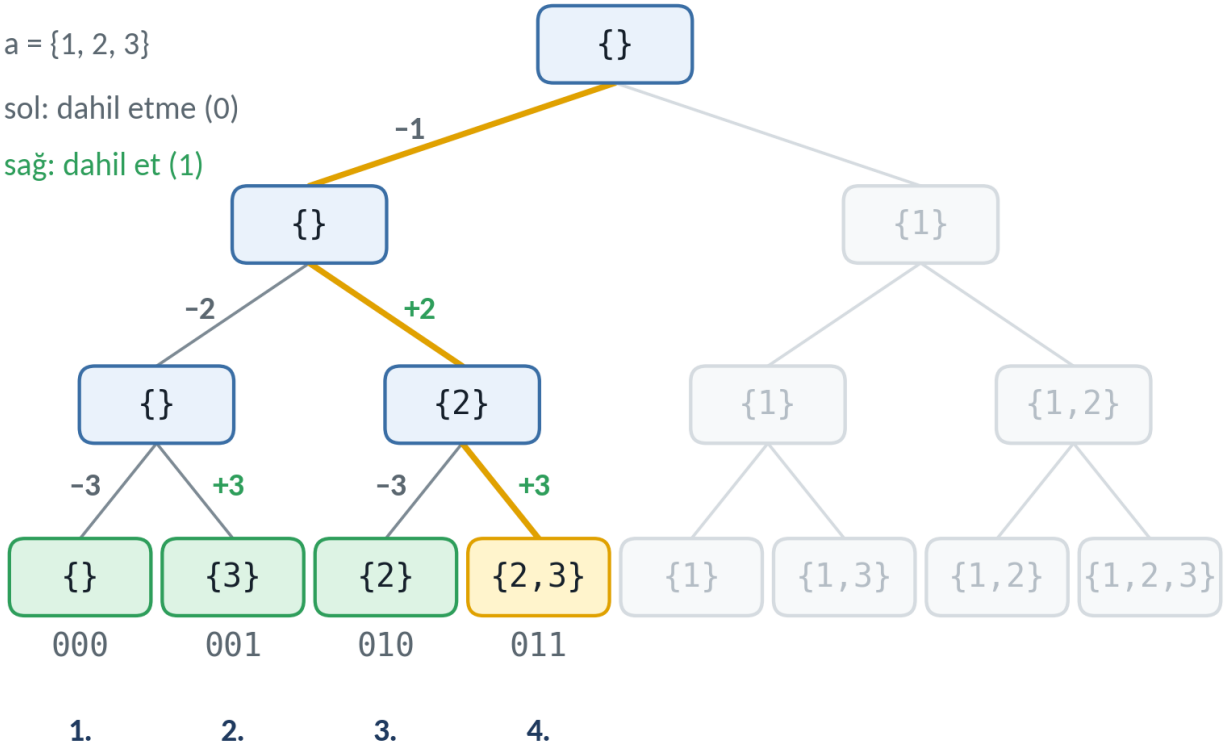
yol (yığın)

çağrı: 4 / 15

uzunluk = 0

- **Problem:** a = {1, 2, 3} → bütün $2^3 = 8$ alt küme, her biri bir kez.
- **Fikir:** sol dal “a[i] alınmaz”, sağ dal “a[i] alınır”.
- Seçilenler bir yığında (`yol`): sağ dala girerken ekle (**seç**), dal bitince çıkar (**geri al**).
- Şimdiye kadar 4 çağrı: kökten ilk yaprağa ({}) inildi.

Sizce sırada ne olacak?



yol (yığın)

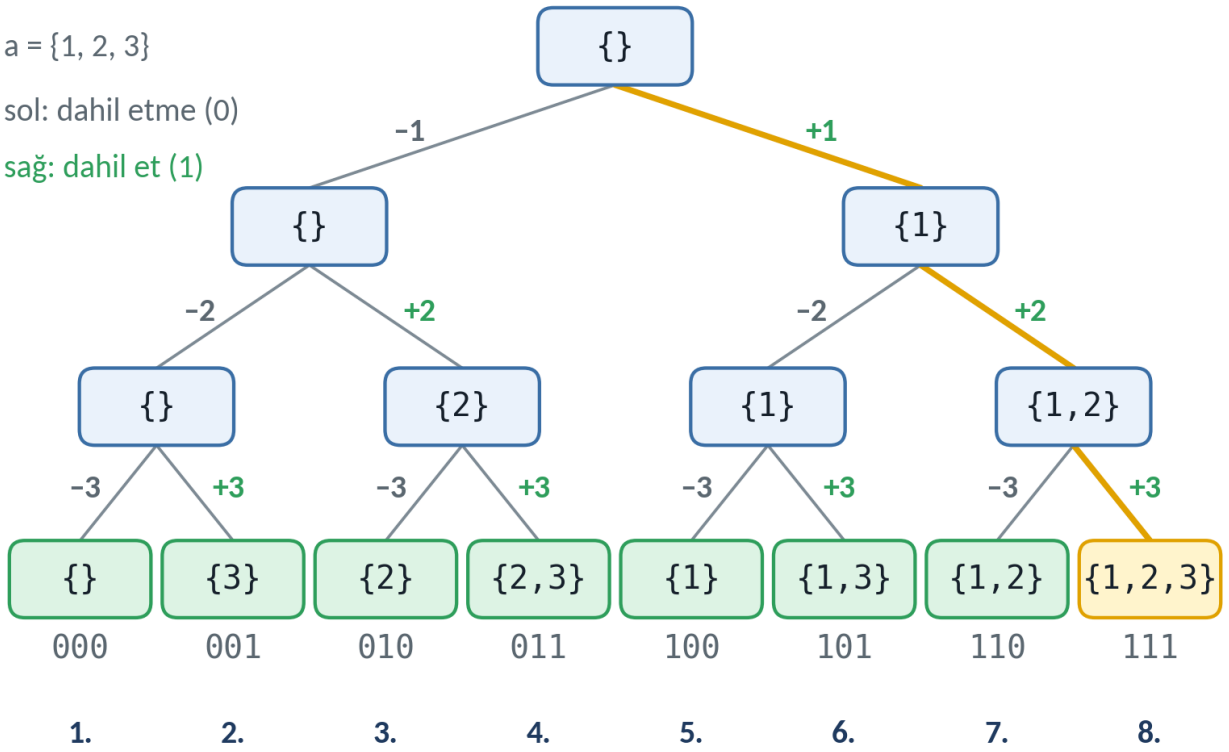
`çağrı: 8 / 15`



`uzunluk = 2`

- Yapraklar {}, {3}, {2}, {2, 3} yazıldı; `yol` = [2, 3].
- **Tahmin edin:** şimdi hangi çerçeveler biter, yığından hangi elemanlar çıkar? 5. küme hangisi?
- İpucu: önce **sol** (alma) dal, sonra **sağ** (al) dal gezilir.

Tam ağaç: 15 çağrı, 8 yaprak, derinlik n + 1



yol (yığın)

çağrı: 15 / 15



uzunluk = 3

- Sıra: {} {3} {2} {2, 3} {1} {1, 3} {1, 2} {1, 2, 3}.
- Karar dizisi $b_0b_1b_2$ ikili sayma sırasıdır (a[0] soldaki bit).
- n = 16: 65 536 küme, 131 071 çağrı, 524 288 eleman (Örnek 4).
- Analiz: $\Theta(n \cdot 2^n)$ — çıktının boyutu kadar; **daha hızlısı olamaz**.

Dahil et / etme: sözde kod ve C

```
1  ALT-KÜMELER(i)
2    eğer i = n ise
3      yol'u yaz; döndür
4    ALT-KÜMELER(i + 1)     ▷ alma
5    yol'a a[i] ekle        ▷ seç
6    ALT-KÜMELER(i + 1)     ▷ dene
7    yol'dan a[i]'yi çıkar ▷ geri al
```

c/H03_04_alt_kumeler.c · satır 38–49 — C

```c
38  static void alt_kumeler(int i)
39  {
40      cagri_sayisi++;
41      if (i == n) {          /* yaprak */
42          kume_isle();
43          return;
44      }
45      alt_kumeler(i + 1);    /* a[i] yok */
46      yol[uzunluk++] = a[i]; /* seç */
47      alt_kumeler(i + 1);    /* dene */
48      uzunluk--;             /* geri al */
49  }
```

Değişmez: çağrının başında `yol`, a[0..i−1] için verilmiş kararlarda seçilenleri tam olarak içerir; dönüşte eski hâline döner.

ALGORİTMA KİMLİK KARTI

Alt küme üretimi *subset generation (power set)*

STRATEJİ	GİRDİ ÖNKOŞULU	EN İYİ	ORTALAMA	EN KÖTÜ	EK BELLEK
Kaba kuvvet: dahil et / etme	Ayrık elemanlar; $n \lesssim 25$	$\Theta(n \cdot 2^n)$	$\Theta(n \cdot 2^n)$	$\Theta(n \cdot 2^n)$	$O(n)$ yığın

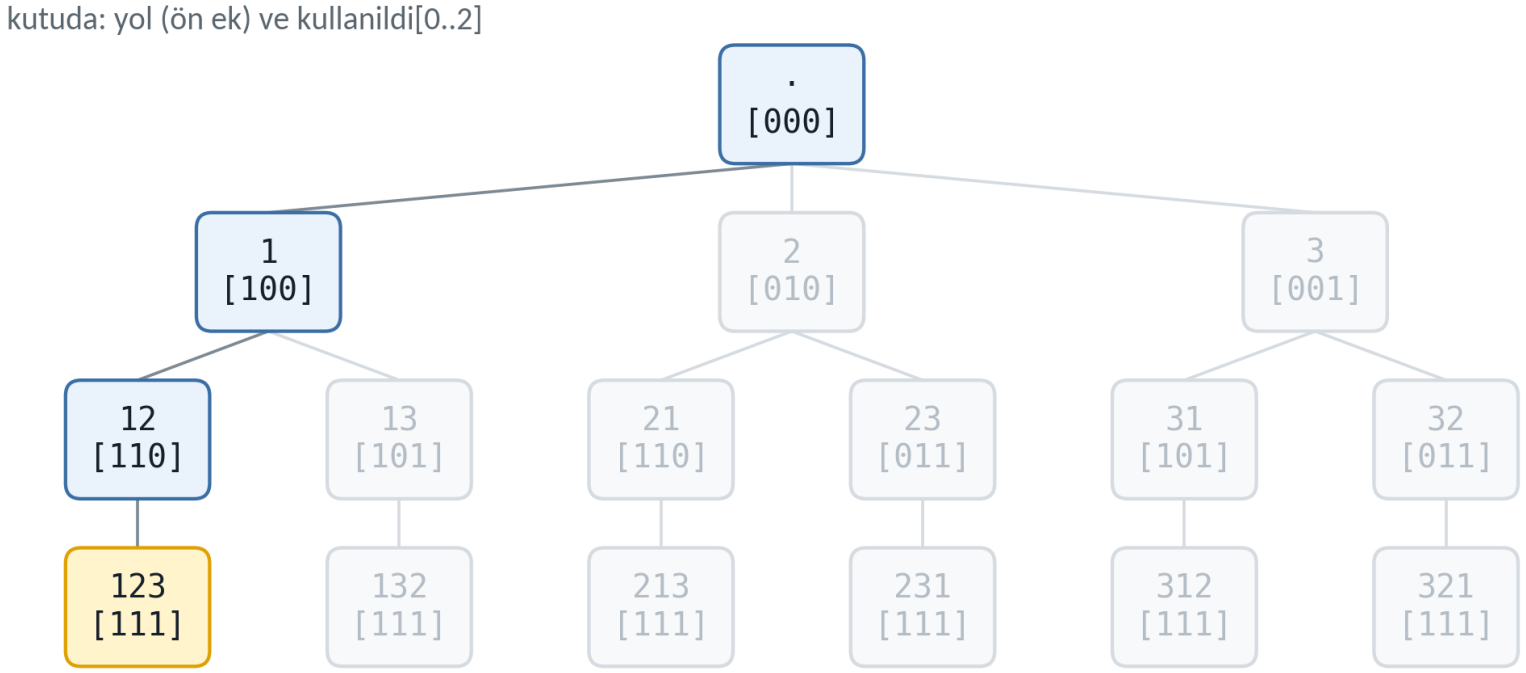
Özellikler	Çıktı boyutu kadar iş: 2^n küme, toplam $n \cdot 2^{n-1}$ eleman; daha hızlısı olamaz. Ağaçta 2^n yaprak ve $2^{n+1} - 1$ çağrı; özyineleme derinliği $n + 1$.
Değişmez / doğruluk	alt_kumeler(i) çağrısında yol, a[0..i−1] için verilmiş kararlarda seçilen elemanları tam olarak içerir; çağrı döndüğünde yol, çağrıdan önceki hâline döner (geri al)
Kütüphane karşılığı	C: yok (elle; bit maskesi döngüsü) · C++: yok (`std::bitset` yalnızca gösterim) · Python: `itertools.product` (yalnızca kâhin)
Ne zaman kullanılır	Tüm seçeneklerin denenmesi gereken küçük problemlerde, daha hızlı algoritmaların kâhini olarak, alt küme toplamı / sırt çantası gibi problemlerin ilk doğru çözümü olarak
Ne zaman kullanılmaz	n büyükken (n = 40'ta $2^{40} \approx 1{,}1 \cdot 10^{12}$ küme); yalnızca koşulu sağlayan kümeler aranıyorsa budamalı geri izleme, en iyi küme aranıyorsa dinamik programlama (H13)

Bit maskesi: aynı aile, döngüyle

maske	ikili $b_2b_1b_0$	a[0] ↔ soldaki bit (b_2) = Örnek 4'ün sırası	a[0] ↔ sağdaki bit (b_0) aynı aile, farklı sıra
0	**000**	{}	{}
1	**001**	{3}	**{1}**
2	**010**	{2}	{2}
3	**011**	{2, 3}	**{1, 2}**
4	**100**	{1}	**{3}**
5	**101**	{1, 3}	{1, 3}
6	**110**	{1, 2}	**{2, 3}**
7	**111**	{1, 2, 3}	{1, 2, 3}

- maske = 0 … $2^n - 1$; k. bit 1 ise a[k] kümede.
- a[0] **soldaki** bite eşlenirse sıra Örnek 4'le aynı; sağdaki bite eşlenirse aile aynı, sıra farklı (001: {3} ya da {1}).
- Program iki yöntemi n = 1 … 16 için karşılaştırır: **kâhinle test**.
- Sınır `1u << n` (işaretsiz): `1 << n` bir `int`'tir, n ≥ 31'de tanımsız davranış (Hata Avcısı 6).

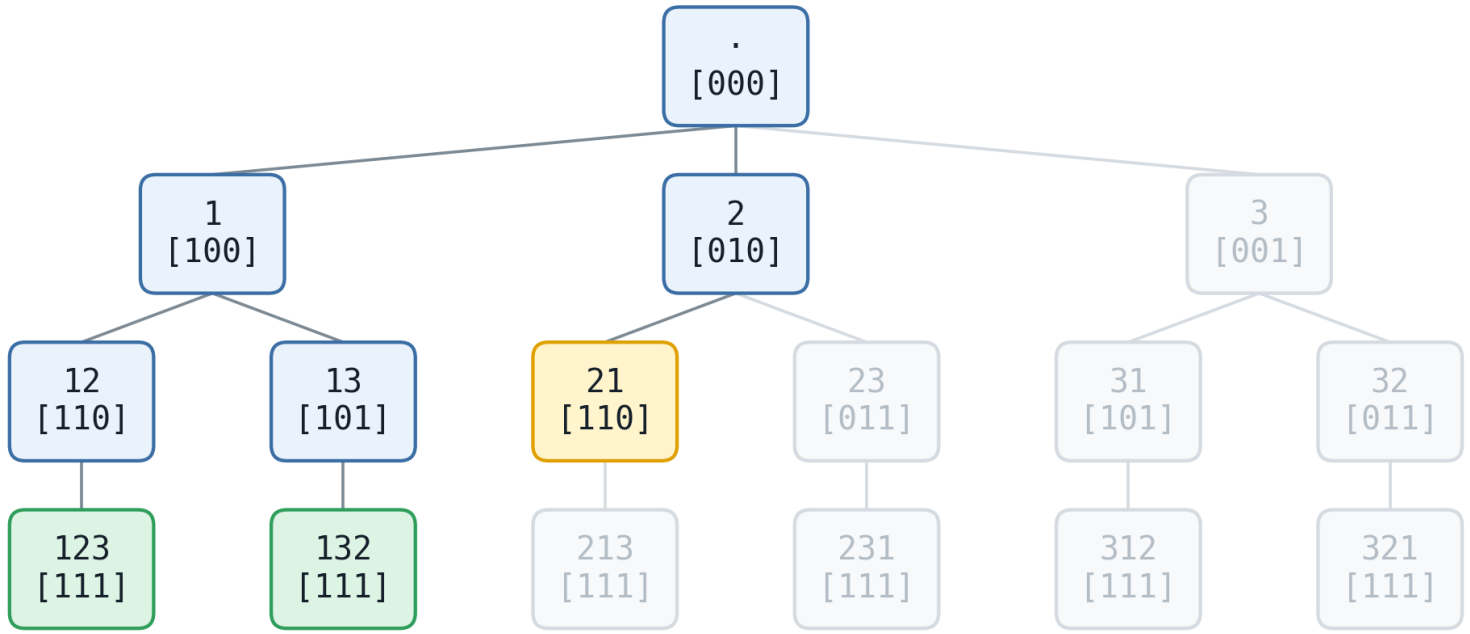
Permütasyon üretimi: konum konum seç



- **Problem:** 1, 2, 3 → bütün 3! = 6 sıralama, **sözlük sırasında**.
- **Fikir:** k. konuma kullanılmamış her elemanı sırayla koy, k + 1'i dene, sonra kaldır.
- `kullanildi` dizisi kökten o düğüme kadar seçilenleri işaretler.
- İlk yaprak: 123.

Sizce sırada ne olacak?

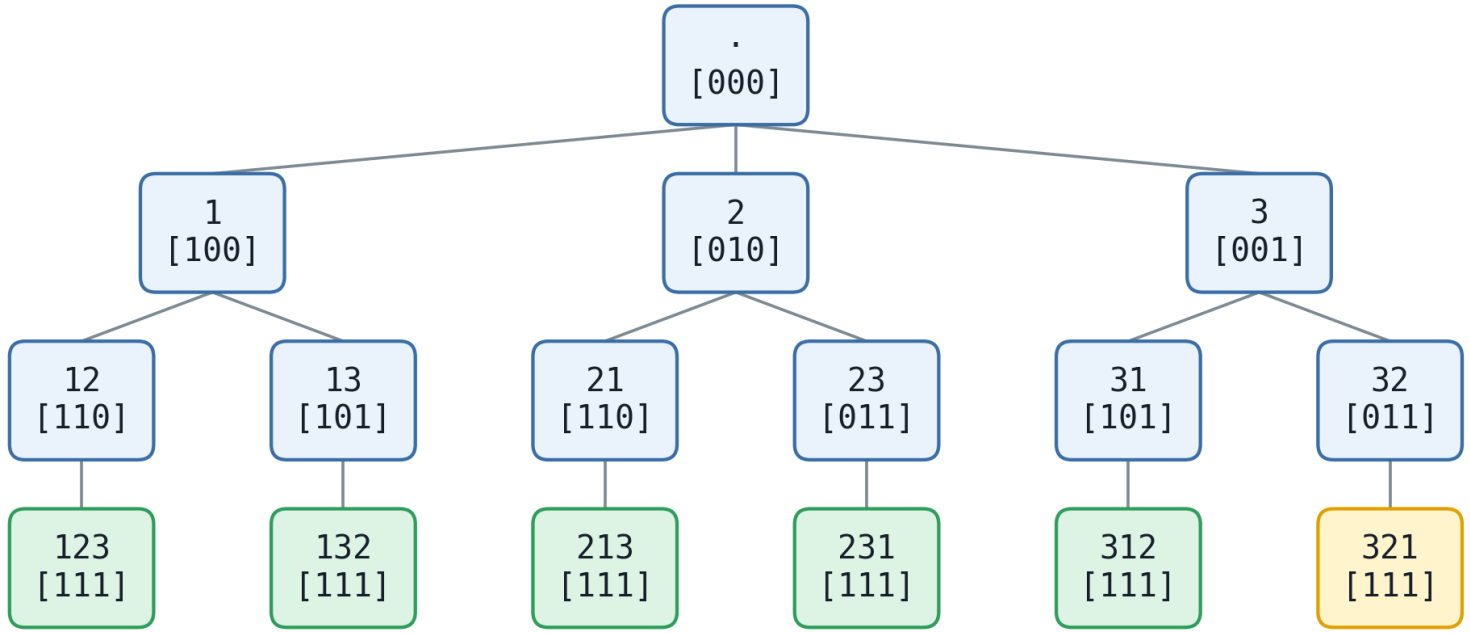
kutuda: yol (ön ek) ve kullanildi[0..2]



- 123 ve 132 yazıldı.
- **Tahmin edin:** `kullanildi` hangi adımlardan geçer, üçüncü permütasyon hangisi?
- İpucu: bir dal bitince işaret **kalkar** (geri al).

Tam ağaç: ⌊e·n!⌋ çağrı

kutuda: yol (ön ek) ve kullanildi[0..2]



- n = 3: 1 + 3 + 6 + 6 = **16** çağrı, 6 yaprak.
- n = 8: 109 601 çağrı / 40 320 = 2,7183 → e.
- Takasla üretim (alttaki şerit) ek dizi kullanmaz ama sözlük sırasını bozar (321, 312).
- Analiz: Θ(n·n!); derinlik n + 1.

Seç – dene – geri al: sözde kod ve C

```
1  PERMÜTASYON(k)
2    eğer k = n ise
3      yol'u yaz; döndür
4    her i = 0 … n−1 için
5      eğer kullanildi[i] = 0 ise
6        kullanildi[i] ← 1   ▷ seç
7        yol[k] ← a[i]
8        PERMÜTASYON(k + 1)  ▷ dene
9        kullanildi[i] ← 0   ▷ geri al
```

c/H03_06_permutasyonlar.c · satır 52–63

C

```c
52      if (k == n) {           /* yaprak */
53          perm_isle(yol);
54          return;
55      }
56      for (int i = 0; i < n; i++) {
57          if (!kullanildi[i]) {
58              kullanildi[i] = 1; /* seç */
59              yol[k] = a[i];
60              permutasyon(k + 1); /* dene */
61              kullanildi[i] = 0; /* geri al */
62          }
63      }
```

Değişmez: `kullanildi[i]` = 1 ⇔ a[i], yol[0..k−1] içinde. Adaylar artan sırada → çıktı sözlük sırasında.

ALGORİTMA KİMLİK KARTI

Permütasyon üretimi *permutation generation*

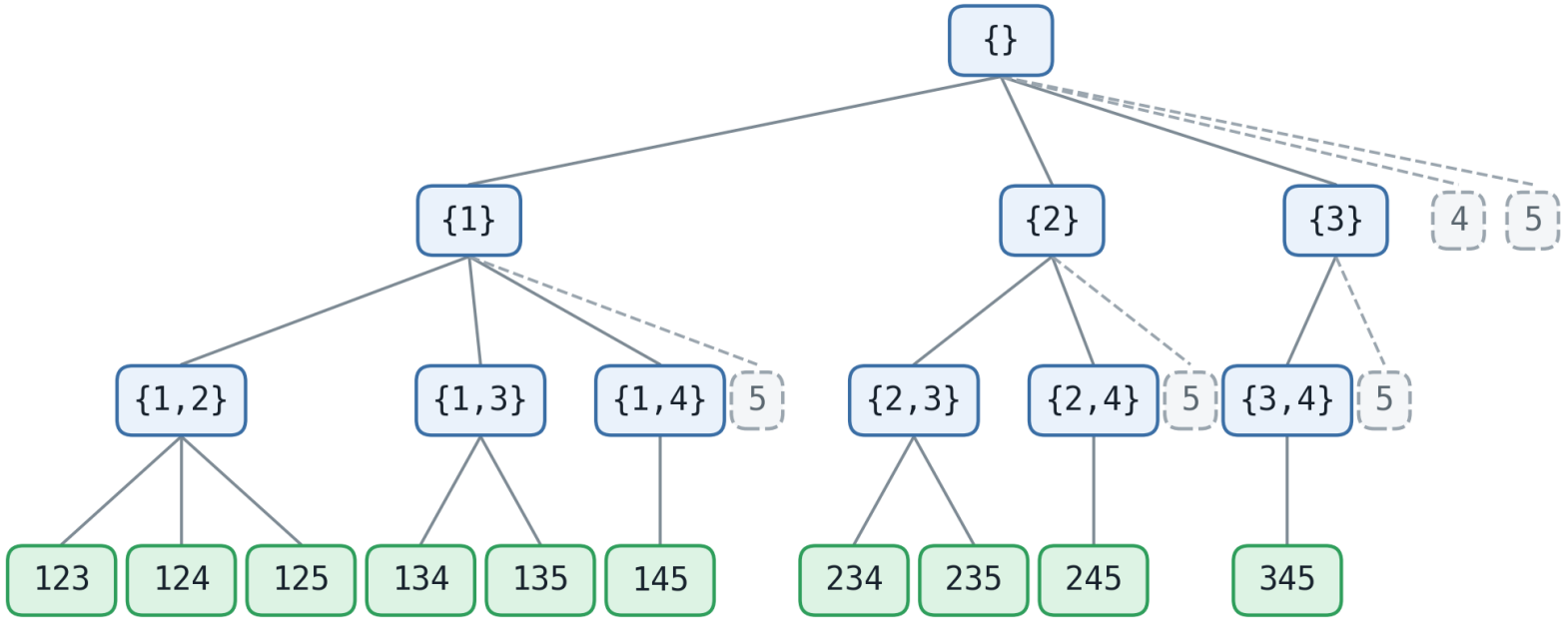
STRATEJİ	GİRDİ ÖNKOŞULU	EN İYİ	ORTALAMA	EN KÖTÜ	EK BELLEK
Kaba kuvvet: seç → dene → geri al	Sözlük sırası için artan girdi	Θ(n·n!)	Θ(n·n!)	Θ(n·n!)	O(n)

Özellikler	Özyineleme derinliği n + 1; toplam çağrı n!/0! + n!/1! + … + n!/n! = ⌊e·n!⌋ ≈ 2,718·n! (n = 8: 40 320 permütasyon, 109 601 çağrı).
Değişmez / doğruluk	permutasyon(k) çağrısında yol[0..k−1] birbirinden farklı elemanlardır ve kullanildi[i] = 1 ⇔ a[i] yol[0..k−1] içindedir
Kütüphane karşılığı	C: yok (elle) · C++: `std::next_permutation` (sözlük sırasında sonraki) · Python: `itertools.permutations(a)` (yalnızca kâhin)
Ne zaman kullanılır	n ≤ ~10–11 için tüm sıralamaların denenmesi (sıralama/çizelgeleme kaba kuvveti, kâhin testleri)
Ne zaman kullanılmaz	n ≥ 12 civarında (12! ≈ 4,8·10⁸ permütasyon); yalnızca sayı gerekiyorsa n! formülü yeter

ÖRNEK 7

k-kombinasyonlar: başlangıç indisi

kutuda: seçilenler · gri kesikli: döngü sınırı i ≤ n − (k − uzunluk) yüzünden açılmayan dal

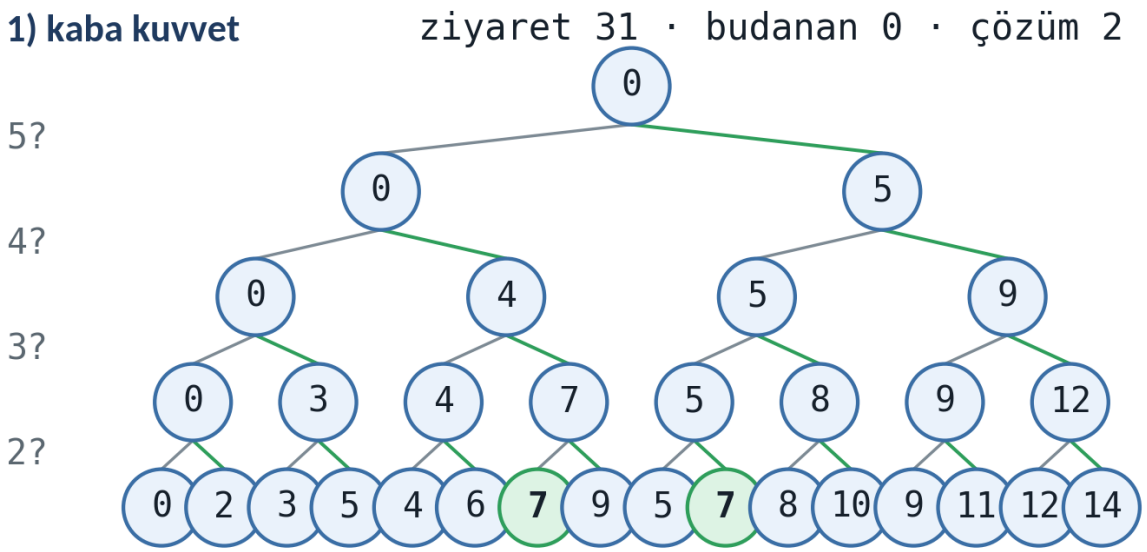


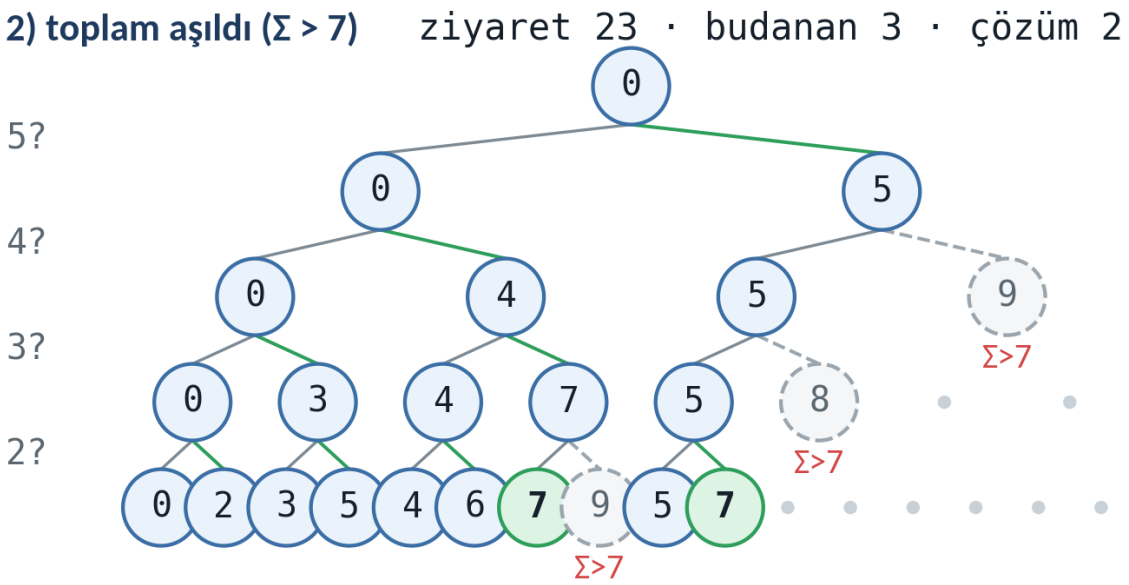
20 çağrı = C(6, 3) · 10 yaprak = C(5, 3)

- `kombinasyon(bas, k)` yalnızca a[bas..n − 1] arasından seçer → indisler hep artar, aynı küme bir kez.
- Döngü sınırı `n − (k − uzunluk)`: yetmeyecek dallar açılmaz (kesikli gri).
- n = 5, k = 3: **20** çağrı, 10 yaprak = C(6, 3).
- Çağrı sayısı her satırda C(n + 1, k): n = 20, k = 10 → 352 716.

Alt küme toplamı: aşılan dalı kes

Problem: pozitif a = {5, 4, 3, 2}, hedef = 7 → toplamı hedefe eşit bütün alt kümeler ({4, 3} ve {5, 2}). Kaba kuvvet 31 düğümün hepsini ziyaret eder.





Sizce budanır mı? Hedef 9 olsaydı "toplam aşıldı" kuralı {5, 4} düğümünü (Σ = 9) budar mıydı? `>` yerine `>=` yazsaydık ne olurdu?

Budamalı geri izleme: sözde kod

```
1  AKT(i, toplam)
2    eğer toplam > hedef ise    ▷ BUDA
3      budanan++; döndür
4    eğer i = n ise
5      eğer toplam = hedef ise
6        çözümü kaydet
7      döndür
8    AKT(i + 1, toplam)
9    AKT(i + 1, toplam + a[i])
```

- Toplam **parametre** olarak taşınır: düğüm başına O(1) iş, geri alma kendiliğinden.
- **Budama güvenli mi?** toplam > hedef ise torunlar bu değere ≥ 0 sayılar ekler → hepsi > hedef.
- Kanıtın varsayımı: **her a[i] ≥ 0**. {5, 4, −3, 2}, hedef 6 → çözüm kaçar (Hata Avcısı 4).
- n = 18 (tohum 42): tam ağaç 524 287; hedef 50 → 3 967 düğüm.

KOD

Kaba kuvvetten tek fark: dört satır

c/H03_08_alt_kume_toplami.c · satır 60–74

C

```c
        ziyaret++;
        if (toplam > hedef) {
            budanan++;                          /* torunların hepsi de aşar */
            return;
        }
        if (i == n) {
            if (toplam == hedef) {
                cozum_isle();
            }
            return;
        }
        budamali(i + 1, toplam);
        yol[uzunluk++] = a[i];
        budamali(i + 1, toplam + a[i]);
        uzunluk--;
```

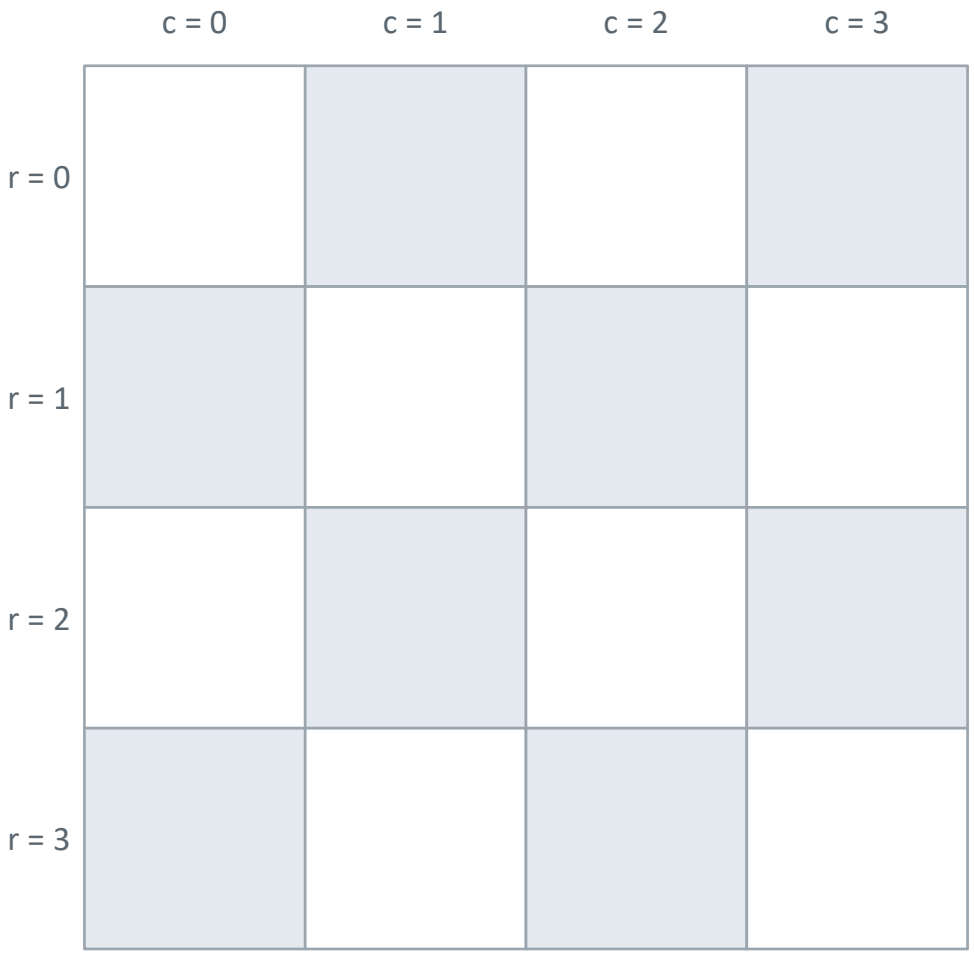
ALGORİTMA KİMLİK KARTI

Alt küme toplamı (geri izleme) *subset sum (backtracking)*

STRATEJİ	GİRDİ ÖNKOŞULU	EN İYİ	ORTALAMA	EN KÖTÜ	EK BELLEK
Geri izleme (budamalı)	Bütün a[i] ≥ 0	—	—	$O(2^n)$	$O(n)$

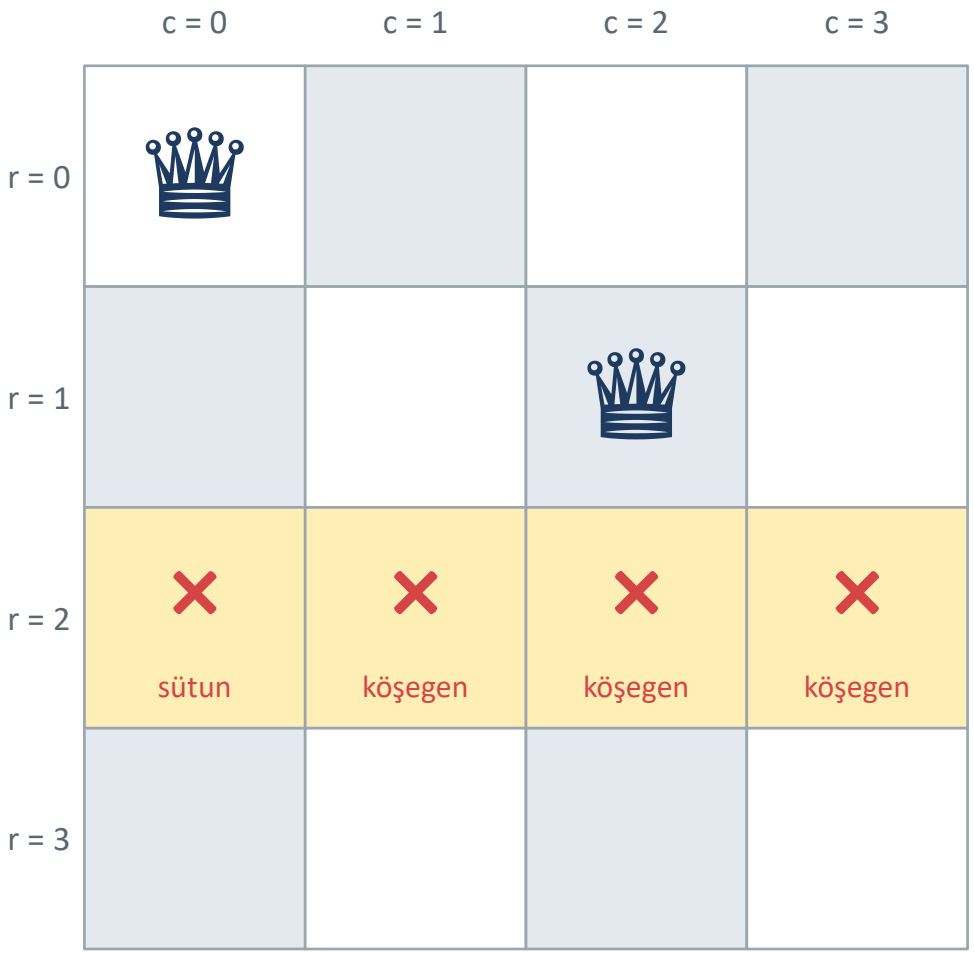
Özellikler	Toplam parametreyle taşındığı için düğüm başına O(1) iş. Bu makinede n = 18, hedef 50: kaba kuvvet 524 287, “toplam aşıldı” 3 967, sıralı + iki sınır 937 düğüm.
Değişmez / doğruluk	Her çağrıda toplam, yol'daki (a[0..i−1] arasından seçilmiş) elemanların toplamına eşittir
Kütüphane karşılığı	C: yok · C++: yok · Python: yok (kâhin: bit maskesiyle 0 … 2^n − 1 taraması)
Ne zaman kullanılır	n ≤ ~25–30 iken tüm çözümler isteniyorsa; hedef toplama göre küçükse budama çok etkilidir
Ne zaman kullanılmaz	Sayılar ve hedef küçük tamsayılarsa O(n·hedef) dinamik programlama (H13); negatif sayı varsa “toplam aşıldı” budaması

N-vezir: N × N tahtaya birbirini tehdit etmeyen N vezir



- Satır başına tam bir vezir → yerleşim `sutun[r]` dizisi.
- N = 4 arama uzayı: C(16, 4) = 1 820 kare seçimi; 4^4 = 256; 4! = 24.
- Geri izleme: **17** düğüm, **44** budanan aday, **2** çözüm.
- Satır satır ilerle; her c'yi soldan sağa dene; aynı sütun ya da köşegendeki aday **budanır**; satırda yer yoksa bir önceki satıra **dön**.

(0, 0) ve (1, 2) kondu: 2. satırın dört sütunu da budandı



ziyaret edilen düğüm: 3

budanan aday: 6

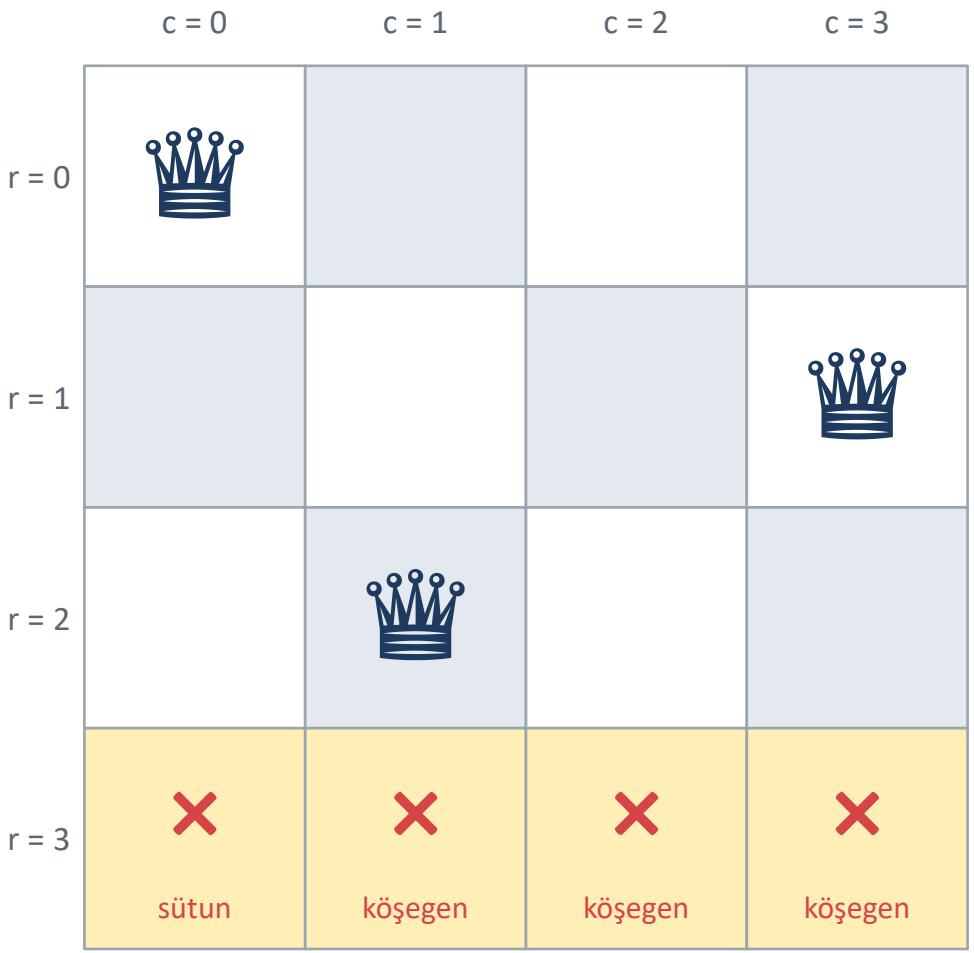
SON OLAYLAR

- (1, 2) yerleştir
- (2, 0) buda: sütun: satır 0, sütun 0
- (2, 1) buda: köşegen: satır 1, sütun 2
- (2, 2) buda: köşegen: satır 0, sütun 0
- (2, 3) buda: köşegen: satır 1, sütun 2

SİZCE SIRADA NE OLACAK?

2. satırda güvenli kare yok. Algoritma şimdi ne yapar? Hangi vezir kalkar, hangi kare denenir?

(1, 3) ve (2, 1) kondu: 3. satırda yine yer yok



ziyaret edilen düğüm: 5

budanan aday: 11

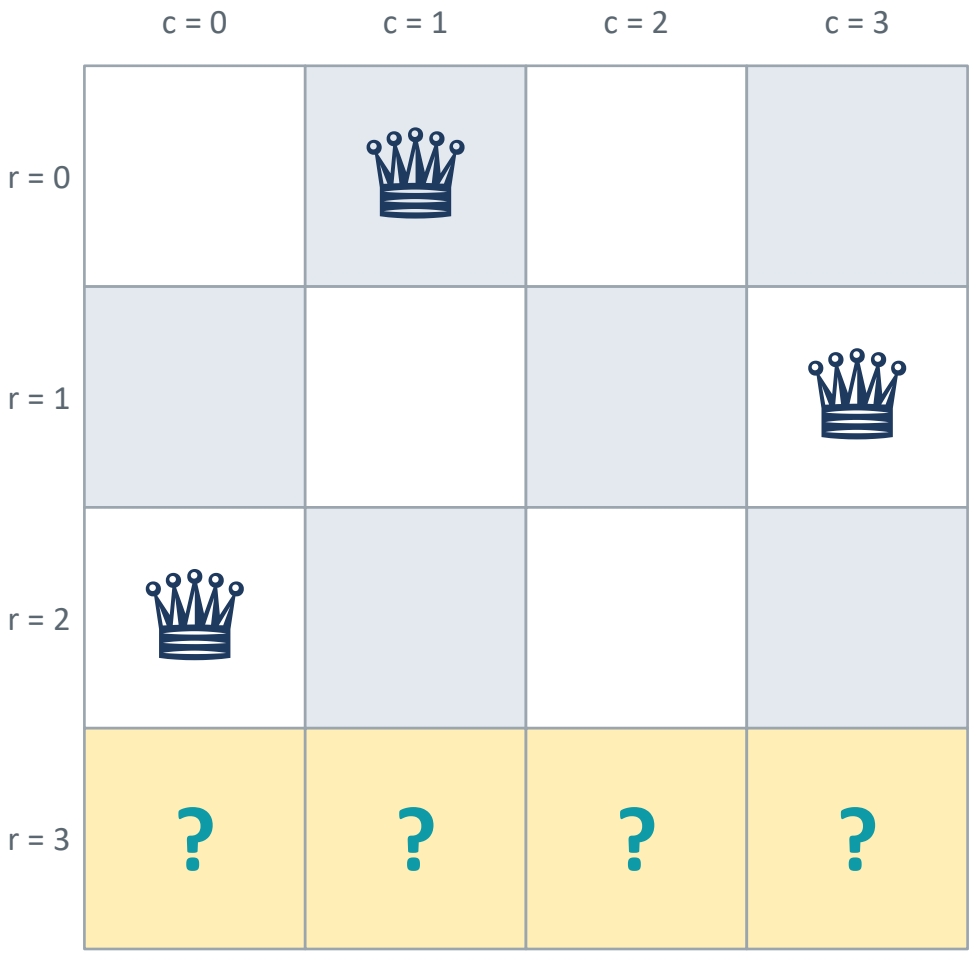
SON OLAYLAR

- (2, 1) yerleştir
- (3, 0) buda: sütun: satır 0, sütun 0
- (3, 1) buda: köşegen: satır 1, sütun 3
- (3, 2) buda: köşegen: satır 2, sütun 1
- (3, 3) buda: köşegen: satır 0, sütun 0

SİZCE SIRADA NE OLACAK?

3. satırın dört sütunu da budandı. Kaç adım geri gidilir? (0, 0) ile başlayan dal kurtarılabilir mi?

(0, 1), (1, 3), (2, 0) kondu



ziyaret edilen düğüm: 8

budanan aday: 16

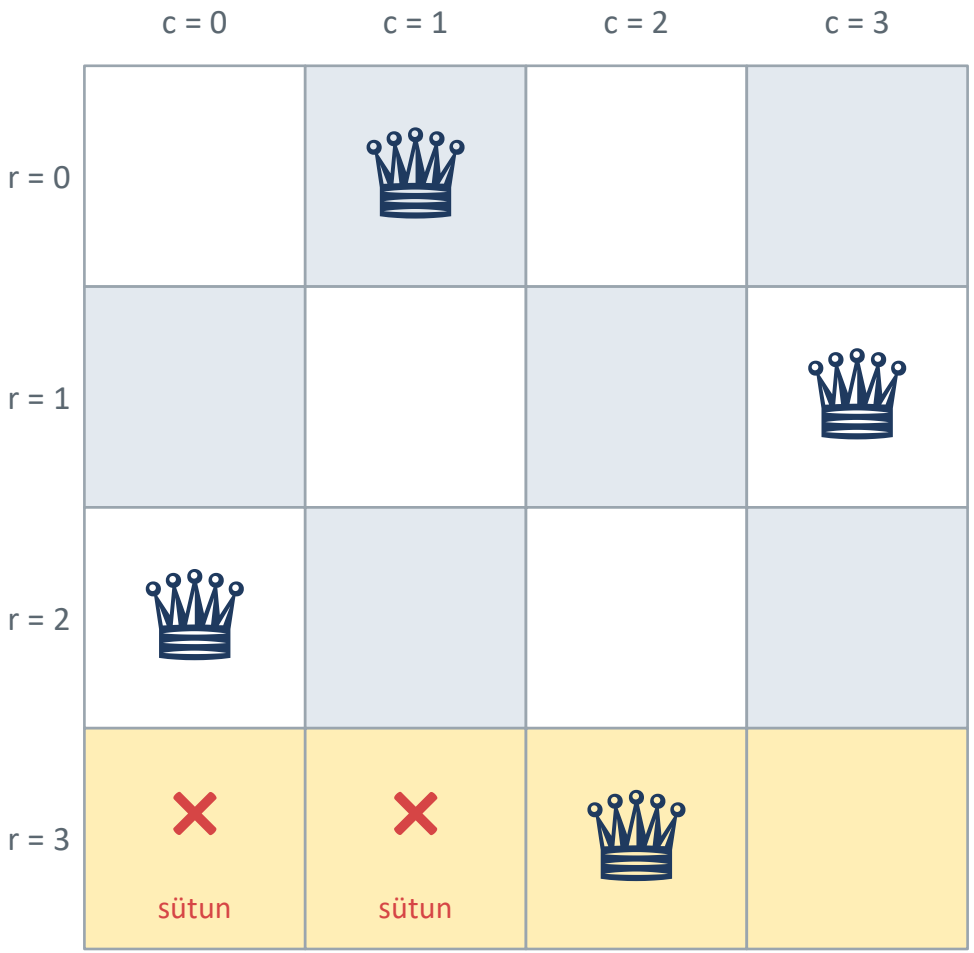
SON OLAYLAR

- (1, 0) buda: köşegen: satır 0, sütun 1
- (1, 1) buda: sütun: satır 0, sütun 1
- (1, 2) buda: köşegen: satır 0, sütun 1
- (1, 3) yerleştir
- (2, 0) yerleştir

TAHMİN ET

Bu sütun güvenli mi? 3. satırda c = 0, 1, 2, 3 için tek tek karar verin: hangisi sütun, hangisi köşegen yüzünden budanır?

İlk çözüm: sütunlar (1, 3, 0, 2)



ziyaret edilen düğüm: 9

budanan aday: 18

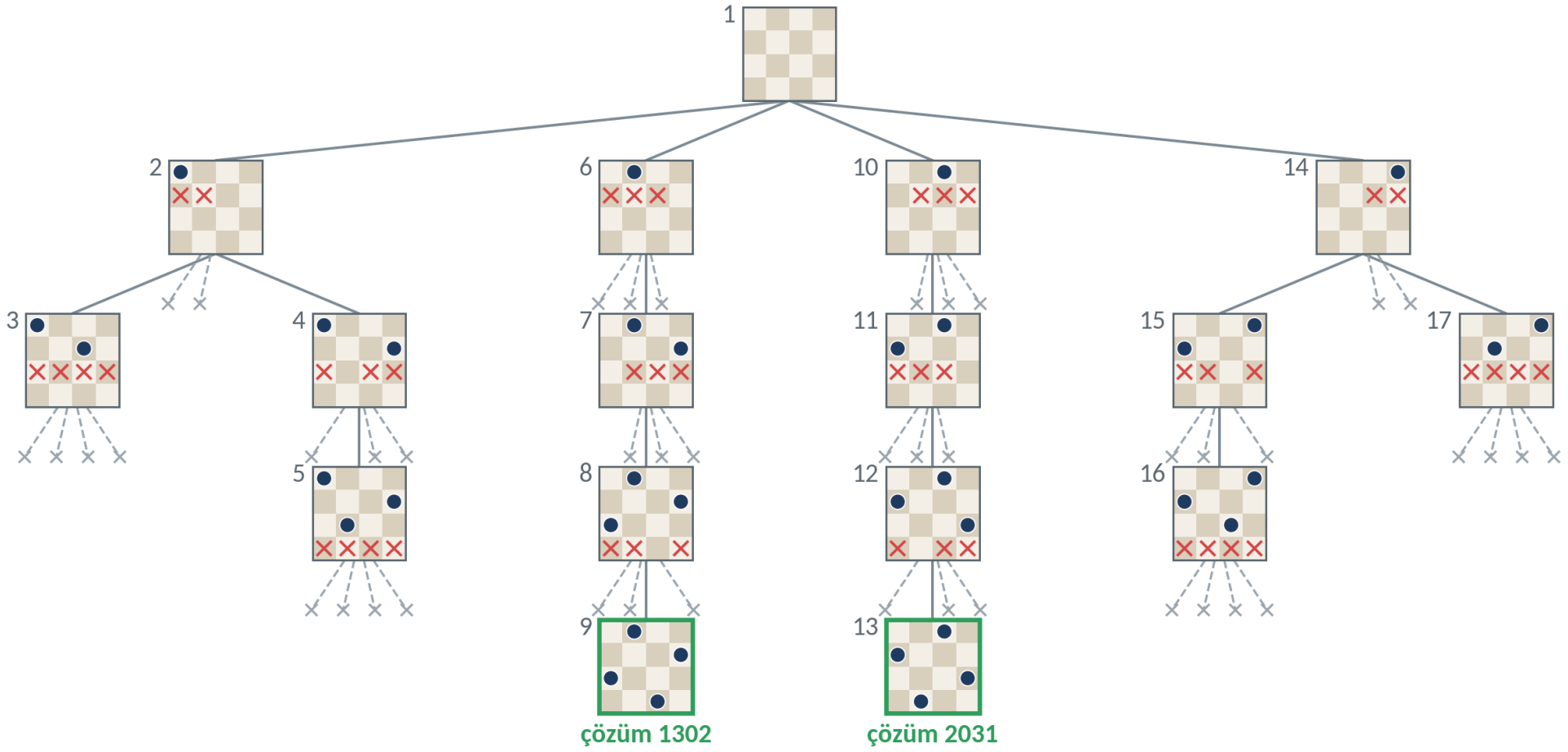
SON OLAYLAR

- (2, 0) yerleştir
- (3, 0) buda: sütun: satır 2, sütun 0
- (3, 1) buda: sütun: satır 0, sütun 1
- (3, 2) yerleştir
- **çözüm:** (1, 3, 0, 2)

ÇÖZÜM

Arama burada durmaz: bütün çözümler isteniyor. Son durum: **17** düğüm, **44** budanan aday, **2** çözüm; ikinci çözüm (2, 0, 3, 1) birincinin aynası.

4-vezir durum uzayı ağacı: 17 düğüm, 44 budanan aday



Numara: ziyaret sırası · kesikli gri: budanan aday (çağrı yapılmaz) · yeşil çerçeve: çözüm

Köşegen indisleri: O(1) güvenlik denetimi

kosegen1[r + c] (↗ yönlü)

	c=0	c=1	c=2	c=3
r=0	0	1	2	**3**
r=1	1	2	●	4
r=2	2	**3**	4	5
r=3	**3**	4	5	6

kosegen2[r − c + n − 1] (↘ yönlü)

	c=0	c=1	c=2	c=3
r=0	3	**2**	1	0
r=1	4	3	●	1
r=2	5	4	3	**2**
r=3	6	5	4	3

(1, 2) vezirinin köşegenleri: r + c = 1 + 2 = 3 ve r − c + n − 1 = 1 − 2 + 3 = 2 (kalın sayılar)

Aynı köşegendeki kareler aynı sayıyı (aynı rengi) taşır: 2n − 1 = 7 köşegen, indis 0 … 6

- "↗" köşegende **r + c** sabit (0 … 2n − 2).
- "↘" köşegende **r − c** sabit; n − 1 ekle → 0 … 2n − 2.
- Üç dizi: `sutun_dolu[c]`, `kosegen1[r + c]`, `kosegen2[r − c + n − 1]`.
- Önceki r veziri taramak O(r); üç dizi okumak O(1). N = 8: tarama 46 752, dizi 15 720 denetim.

Dizilerle N-vezir: sözde kod ve C

```
YERLEŞTİR(r)
  eğer r = n ise
    çözüm++; döndür
  her c = 0 … n−1 için
    eğer c, r+c, r−c+n−1 boş ise
      üçünü işaretle    ▷ seç
      YERLEŞTİR(r + 1)  ▷ dene
      işaretleri kaldır ▷ geri al
```

c/H03_10_n_vezir_hizli.c · satır 69–83

C

```c
static void yerlestir_dizi(int r)
{
    dugum_sayisi++;
    if (r == n) {
        cozum_sayisi++;
        return;
    }
    for (int c = 0; c < n; c++) {
        if (bos_mu(r, c)) {
            isaretle(r, c, 1); /* seç */
            yerlestir_dizi(r + 1); /* dene */
            isaretle(r, c, 0); /* geri al */
        }
    }
}
```

Değişmez: 0..r−1 satırlarında birbirini tehdit etmeyen birer vezir var ve işaret dizileri tam olarak bunların sütun/köşegenlerini gösterir.

ALGORİTMA KİMLİK KARTI

N-vezir (geri izleme) *N-queens (backtracking)*

STRATEJİ	GİRDİ ÖNKOŞULU	EN İYİ	ORTALAMA	EN KÖTÜ	EK BELLEK
Geri izleme (budamalı)	N ≥ 1 (N = 2, 3 çözümsüz)	—	—	O(n!) üst sınır	O(n)

Özellikler	Budama, ziyaret edilen düğüm sayısını büyük ölçüde azaltır: N = 8'de budamasız ağacın 19 173 961 düğümüne karşı 2 057 düğüm (≈ 9 321 kat), 92 çözüm.
Değişmez / doğruluk	yerlestir(r) çağrısında 0..r−1 satırlarındaki vezirler birbirini tehdit etmez ve sütun / köşegen dizileri tam olarak bu vezirleri işaretler
Kütüphane karşılığı	C: yok · C++: yok · Python: yok (kâhin: `itertools.permutations(range(n))` ile n! sütun düzenini tek tek denemek)
Ne zaman kullanılır	Kısıt sağlama problemlerinde (yerleştirme, çizelgeleme, bulmacalar) tüm çözümler ya da bir çözüm gerektiğinde; tek çözüm yetiyorsa ilk çözümde durulur (Hata Avcısı 2)
Ne zaman kullanılmaz	N büyüdükçe tüm çözümleri saymak hızla uygulanamaz hâle gelir (çözüm sayısının kendisi üstel büyür); budama olmadan hiç kullanılmaz

Üç dilde: alt küme üretimi (seç → dene → geri al)

satır 38–49 — C

```c
static void alt_kumeler(int i)
{
    cagri_sayisi++;
    if (i == n) {          /* yaprak */
        kume_isle();
        return;
    }
    alt_kumeler(i + 1);    /* a[i] yok */
    yol[uzunluk++] = a[i]; /* seç */
    alt_kumeler(i + 1);    /* dene */
    uzunluk--;             /* geri al */
}
```

satır 37–48 — C++

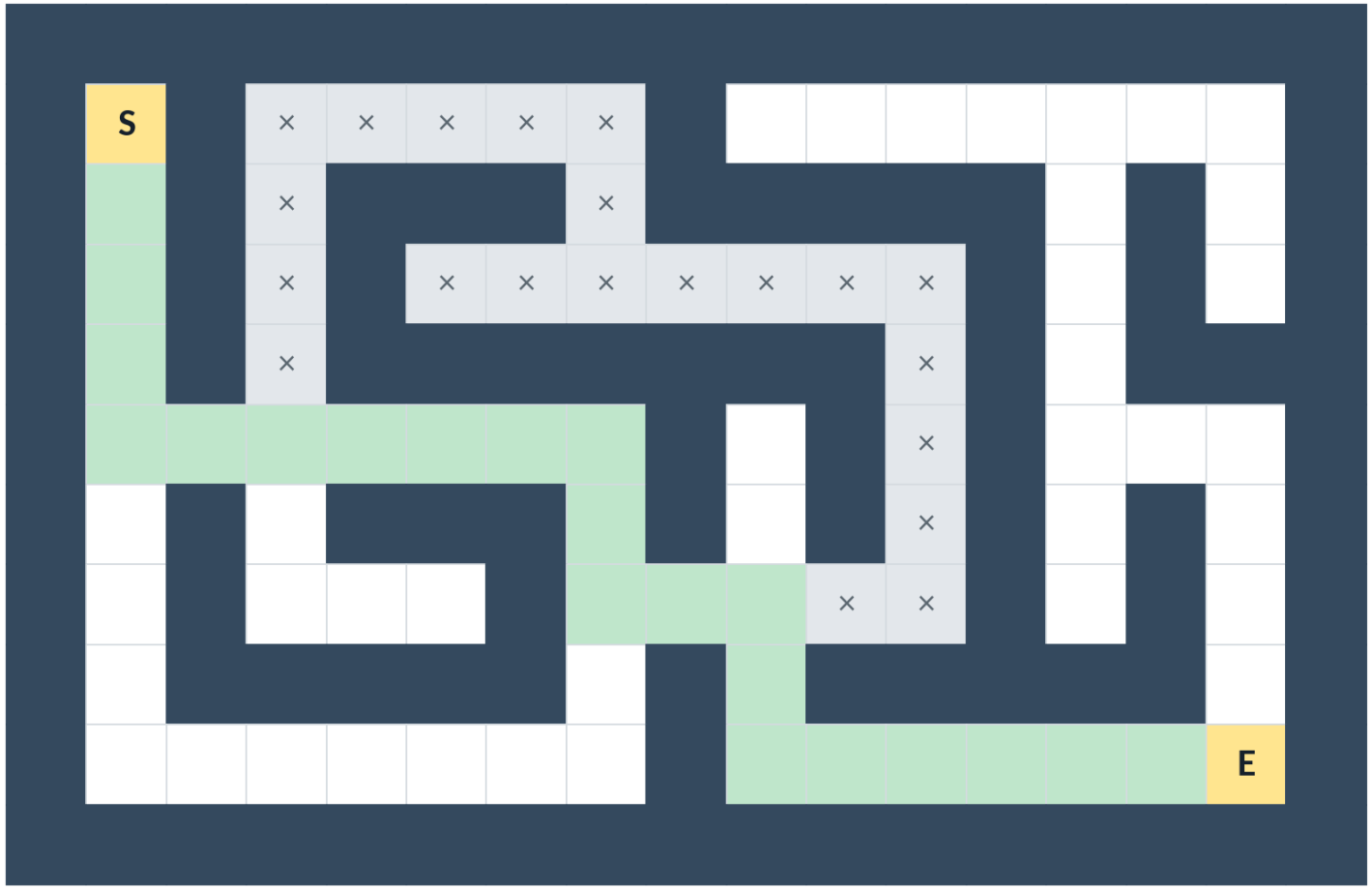
```cpp
static void alt_kumeler(int i)
{
    cagri_sayisi++;
    if (i == n) {          // yaprak
        kume_isle();
        return;
    }
    alt_kumeler(i + 1);    // a[i] yok
    yol.push_back(a[i]);   // seç
    alt_kumeler(i + 1);    // dene
    yol.pop_back();        // geri al
}
```

satır 31–40 — Python

```python
def alt_kumeler(i):
    global cagri_sayisi
    cagri_sayisi += 1
    if i == n:             # yaprak
        kume_isle()
        return
    alt_kumeler(i + 1)     # a[i] yok
    yol.append(a[i])       # seç
    alt_kumeler(i + 1)     # dene
    yol.pop()              # geri al
```

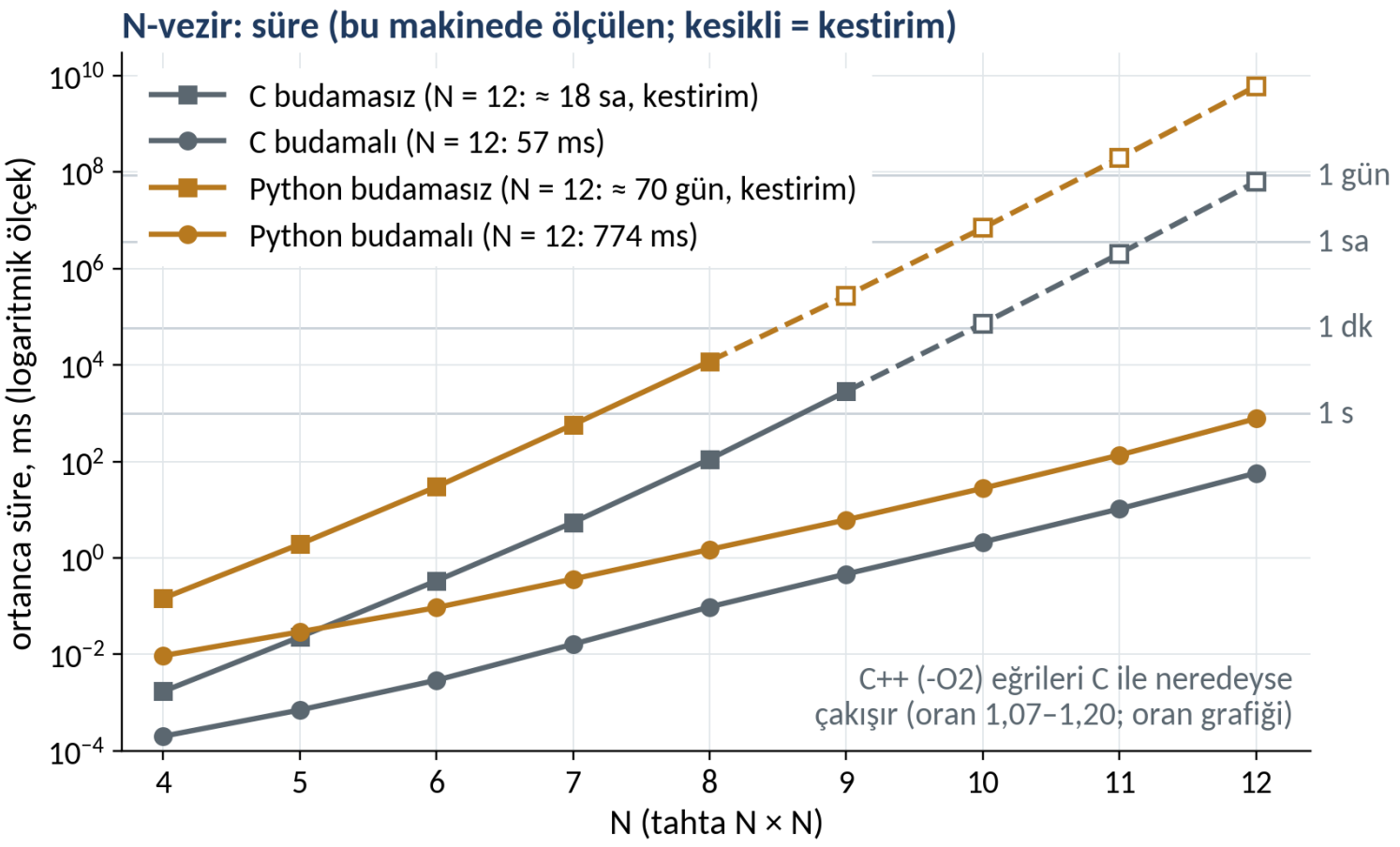
`H03_04_alt_kumeler` (.c · .cpp · .py) — Seçilenler C'de dizi + `uzunluk`, C++'ta `push_back`/`pop_back`, Python'da `append`/`pop`; Python'da sayaç için `global`.

Labirent: ızgarada geri izleme (Örnek 11)



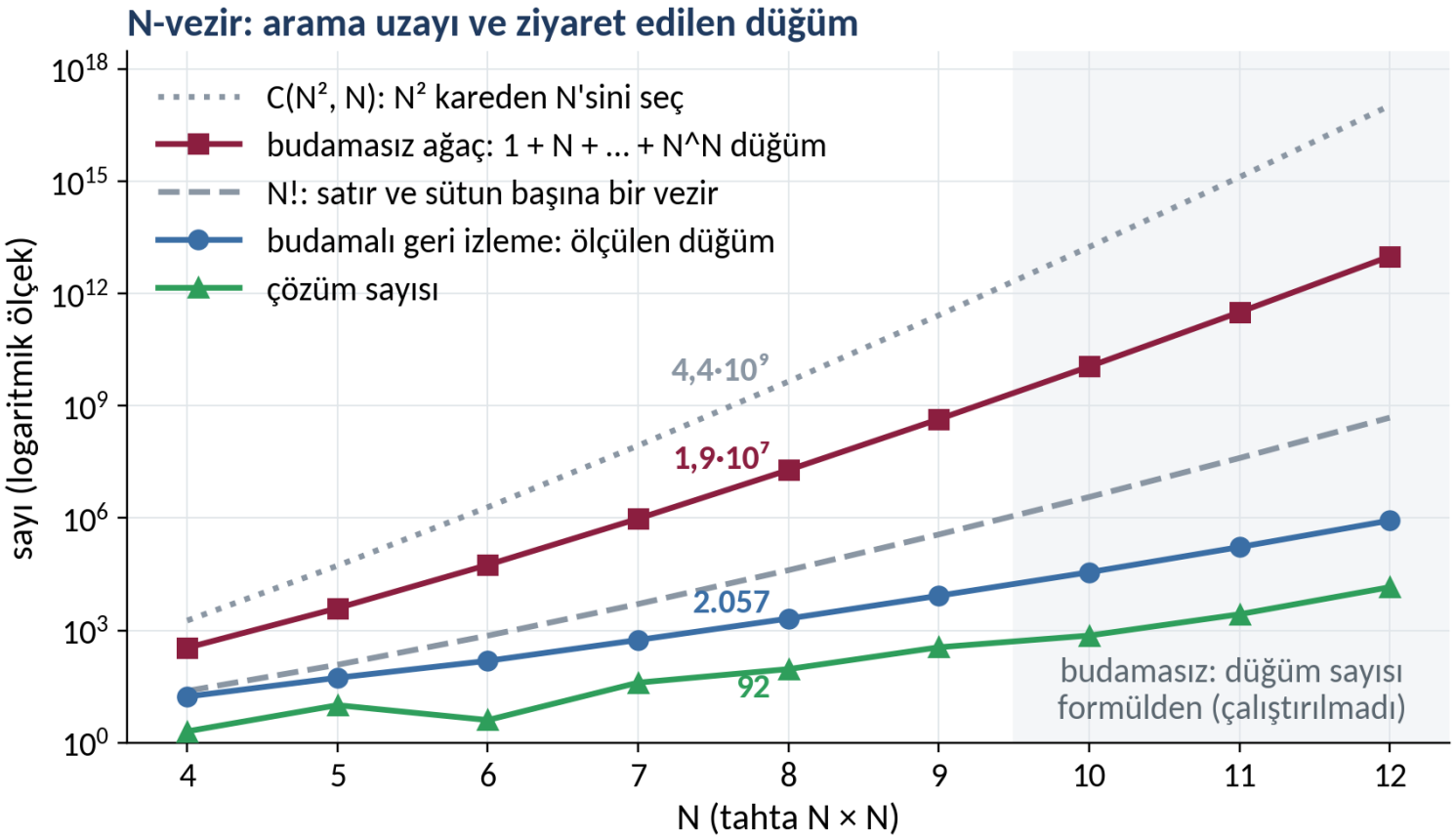
- Durum = hücre; adaylar = 4 komşu (sağ, aşağı, sol, yukarı).
- Gir: * (seç); çıkmaz: x (geri al). İşaretler döngüyü de önler.
- 44 hücreye girildi, 21 hücreden dönüldü; yol 22 adım.
- DFS ilk bulduğu yolu verir; **en kısa olduğu güvence edilmez** (BFS: H11).
- Sudoku (Örnek 13): aynı şablon, satır/sütun/kutu dizileriyle O(1) denetim; 456 082 düğüm, ilk çözümde dur.

Haftanın Deneyi: budama mı, dil mi?



- N = 8: budama düğümleri **9 321 kat** azaltır; Python C'den ≈ 107 kat yavaş.
- Python budamalı (1,50 ms) < C budamasız (111,6 ms).
- N = 12 budamasız C ≈ 17,6 sa (**kestirim**); budamalı 57,4 ms.
- Düzen: 1 ısınma + 5 tekrar, ortanca; kesikli = kestirim.

Düğüm sayıları: makineden bağımsız gerçek



- Budamasız: $1 + N + \ldots + N^N$ ($N = 8$: 19 173 961).
- Budamalı: $N = 8 \rightarrow 2\,057$; $N = 12 \rightarrow 856\,189$.
- $8 \rightarrow 9$ geçişinde budamasız düğüm 22,7 kat, budamalı 4,1 kat büyür.
- Sayaç süreden önce gelir: sayılar her makinede aynı, süreler değil.

Hata Avcısı: yedi hata, gerçek belirtiler

#	Hata	Belirti (bu makinede)	Çözüm
1	Seçimi geri almayı unutmak	6 yerine 1 permütasyon; kullanildi = {1, 1, 1}	her “seç”in simetrik “geri al”ı
2	Tüm çözümler istenirken ilk çözümde dönmek	N = 8: 92 yerine 1 çözüm (114 düğüm)	iki sözleşme, iki fonksiyon
3	Tekrarlı elemanda yinelenen permütasyon	(1, 1, 2): 3 yerine 6 sonuç	sırala + “eşit, soldaki kullanılmadıysa atla”
4	Negatif sayıda “toplam aşıldı” budaması	{5, 4, −3, 2}, hedef 6: 2 yerine 1 çözüm	budamayı yalnız a[i] ≥ 0 iken kullan
5	Eksik temel durum	Python RecursionError; C -O0 SIGSEGV (139)	temel durum n <= 0
6	1 << n (n ≥ 31)	−2147483648, 1; uyarı yok, UBSan bildirir	1ULL << n
7	Paylaşılan durumu kopyalamamak (Python)	sonuc = [[], …, []]	sonuc.append(yol[:])

HA1 · Geri almayı unutmak

hata_avcisi/H03_HA1_geri_alma_unutma.c · satır 25–31

C

```c
25      for (int i = 0; i < N; i++) {
26          if (!kullanildi[i]) {
27              kullanildi[i] = 1;      /* seç */
28              yol[k] = a[i];
29              permutasyon(k + 1);     /* dene */
30                                      /* HATA: geri al unutuldu */
31          }
```

HATALI

- Çıktı: `123`, `permütasyon sayısı = 1 (beklenen 6)`
- `kullanildi = {1, 1, 1}`: işaretler hiç kalkmıyor.

hata_avcisi/H03_HA1_geri_alma_unutma_duzeltilmis.c · satır 25–31

C

```c
25      for (int i = 0; i < N; i++) {
26          if (!kullanildi[i]) {
27              kullanildi[i] = 1;      /* seç */
28              yol[k] = a[i];
29              permutasyon(k + 1);     /* dene */
30              kullanildi[i] = 0;      /* geri al */
31          }
```

DÜZELTİLMİŞ

- 6 permütasyon; sonda {0, 0, 0}.
- Kural: her **seç**in simetrik bir **geri al**ı vardır.

HA7 · Python: listeyi değil başvuruyu saklamak

hata_avcisi/H03_HA7_liste_kopyasi.py · satır 15–22 — Python

```python
def alt_kumeler(i):
    if i == len(a):
        sonuc.append(yol)          # HATA: kopya değil, aynı liste
        return
    alt_kumeler(i + 1)
    yol.append(a[i])
    alt_kumeler(i + 1)
    yol.pop()
```

HATALI

- `sonuc = [[], [], [], [], [], [], [], []]`
- `aynı nesne mi? True`: sekiz başvuru, tek liste.

hata_avcisi/H03_HA7_liste_kopyasi_duzeltilmis.py · satır 14–21 — Python

```python
def alt_kumeler(i):
    if i == len(a):
        sonuc.append(yol[:])       # o anki durumun kopyası
        return
    alt_kumeler(i + 1)
    yol.append(a[i])
    alt_kumeler(i + 1)
    yol.pop()
```

DÜZELTİLMİŞ

- `yol[:]` o anki durumun **kopyası**.
- C++'ta `push_back(yol)` zaten kopyalar: benzer sözdizimi, farklı anlam.

Dil Penceresi: aynı özyineleme, iki farklı sınır

satır 17–23 — C

```c
static long long toplam(long long n)
{
    if (n == 0) {
        return 0;
    }
    return n + toplam(n - 1);
}
```

satır 17–23 — C++

```cpp
static long long toplam(long long n)
{
    if (n == 0) {
        return 0;
    }
    return n + toplam(n - 1);
}
```

satır 15–18 — Python

```python
def toplam(n):
    if n == 0:
        return 0
    return n + toplam(n - 1)
```

`H03_12_ozyineleme_siniri` (.c · .cpp · .py) — C (-O0): `derinlik 10000: toplam = 50005000` · Python: `derinlik 1000: RecursionError yakalandı` — C'de sınır yığındır (Linux'ta tipik olarak SIGSEGV), Python'da bir sayaç.

Dil Penceresi: bu haftanın dört farkı

Üreteç (Python)

- `yield`: kümeler **tembel** üretilir (Kavram 1).
- C/C++17: geri çağırma ya da açık yığın.

Çözümü saklamak

- Python `append(yol)` başvuru ekler → `yol[:]`.
- C++ `push_back(yol)` kopyalar.

Kütüphane = kâhin

- C++ `std::next_permutation`.
- Python `itertools`, `math.comb`.

Bitler

- `vector<bool>`: `v[i]` bir **vekil** nesne.
- `1 << n`: C'de `int` taşar, Python'da sınırsız.

Mühendis Gözüyle: ölç ve kaynakla karşılaştır

876	876	15 720	15 720
Wirth (1971): ilk çözüme dek sınanan kısmi yerleşim	bizim ölçümümüz (HA2 düzeltilmiş, 114 düğüm)	Wirth (1971): 92 çözüm için kare sınaması	bizim ölçümümüz (Örnek 10, dizi denetimi)

- Aynı arama ağacı (Wirth sütun sütun, biz satır satır: tahtanın yansıması) → elli yıl sonra **aynı iki sayı**: ağaç ve sayaçlar doğru yerde.
- **Kaba kuvvet ilk doğru çözümdür ve kâhin olarak saklanır**; arama uzayı kod yazılmadan önce hesaplanır.

PROJE

Mini-OBS 2 v2.2: AKTS aralığına uyan ders kümeleri

- Mini-OBS'ye bir seçmeli ders planlayıcısı ekliyoruz: AKTS toplamı belirli bir aralıkta kalan bütün ders kümelerini geri izlemeyle üretiyor, budamanın kazancını düğüm sayacıyla ve süreyle ölçüyor, sonucu bit maskesi kaba kuvvetiyle doğruluyoruz.
- Başlangıç indisli ağaç (Örnek 7) + iki budama (Örnek 8'in sınırları) + bit maskesi kâhini (Örnek 5).
- 30 derslik havuz, [24, 30] AKTS: üst sınır budaması 1 073 741 824 düğümlük ağacın 1 434 525'ini (%0,13) ziyaret eder; budamasızdan 715 kat hızlı.
- Kabul ölçütleri **düğüm sayılarıyla** (makineden bağımsız) ve kâhinle karşılaştırmayla verilir.

İleri Bakış ve Merak

H6 · Böl ve fethet

Özyineleme ağacı maliyet aracı olur: T(n) = 2T(n/2) + n'de her düzeyin işi toplanır.

H11 · Graf arama

Labirent bir graftır; DFS bu haftanın geri izlemesidir, BFS en kısa yolu bulur.

H13 · Dinamik programlama

fib(5)'teki 9 tekrar memoization ile kalkar; alt küme toplamı ve ders seçimi (0/1 sırt çantası) DP ile.

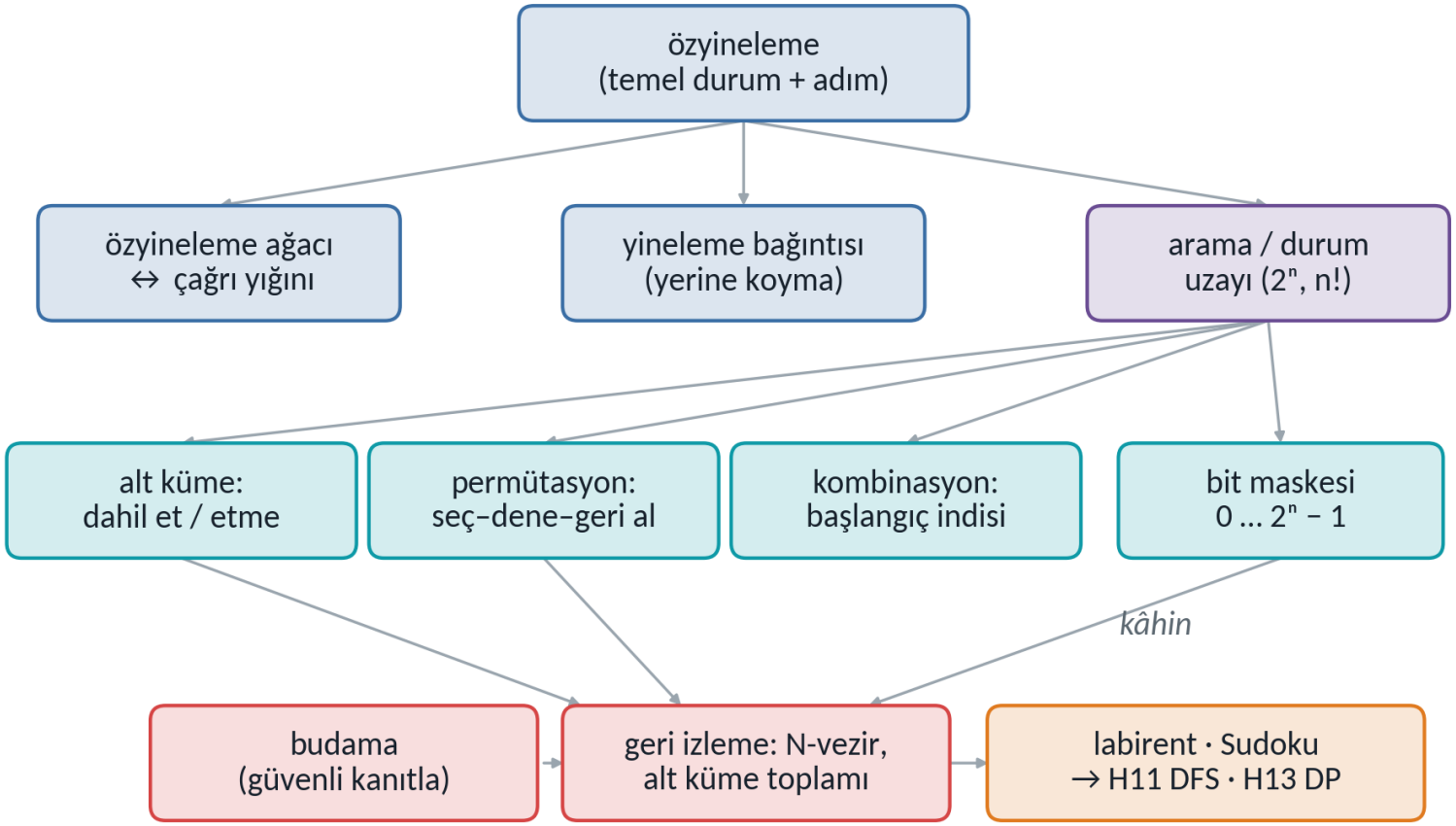
Dal ve sınır · ileri dersler

En iyi değer aranıyorsa dal ve sınır (branch and bound); kısıt programlama: yöneylem araştırması ve yapay zekâ dersleri.

Merak: neden ⌊e·n!⌋?

k derinliğinde n!/(n − k)! düğüm; toplam n!·Σ 1/j! → e. Bit maskeli N-vezir: üç tamsayı, `m & -m`.

Özet: haftanın kavram haritası



- Ağaç = bütün çağrılar (zaman); yığın = bir an (bellek).
- Üç şablon: dahil et/etme · seç–dene–geri al · başlangıç indisi.
- Geri izleme = DFS + **güvenli** budama.
- Uygulanabilirlik: 2^n için $n \approx 27$, $n!$ için $n \approx 11$ (1 s, C).

ÇIKIŞ BİLETİ

Dersten çıkmadan önce üç soru

1. `g(n)` önce g(n − 1)'i çağırır, sonra n'yi yazar, sonra g(n − 2)'yi çağırır; n ≤ 0'da durur. `g(3)` ne yazdırır, kaç çağrı yapılır?

2. Bütün permütasyonları listeleyen bir algoritma neden Θ(n!)'den hızlı olamaz? n = 12'de kaç permütasyon var?

3. Alt küme toplamındaki “toplam > hedef ise dur” budaması hangi girdide çözüm kaçırır? Bir örnek verin.

Hazırlık: bu haftadan sonra

- **Okuma:** ders notu Bölüm 3–4 (ağaç, şablonlar, dört algoritma) ve Bölüm 7 (Hata Avcısı).
- **Alıştırmalar:** 3.3 (yerine koyma), 3.8 (budama hatası), 3.10–3.12 (kod yazma: ardışık 1'siz diziler, rakam kümeleri, N-vezirde simetri).
- **Web sayfası:** özyineleme ağacı, N-vezir (tahmin modu), alt küme/permütasyon üretici ve labirent oynatıcıları; 10 soruluk quiz.
- **Proje:** Mini-OBS 2 v2.2 — kabul ölçütleri düğüm sayılarıyla.
- **Atölyeler (ders notu 11.8–11.10):** 1 arama ağacı çizme (n = 3) · 2 N-vezir yarışı: hangi budama kaç düğüm kazandırır? · 3 bulmaca çözücü (Sudoku ya da kelime arama)
- **Gelecek hafta (H4):** arama algoritmaları; bugünkü özyineleme ağacı ikili aramada tek bir yola iner.